

Organization

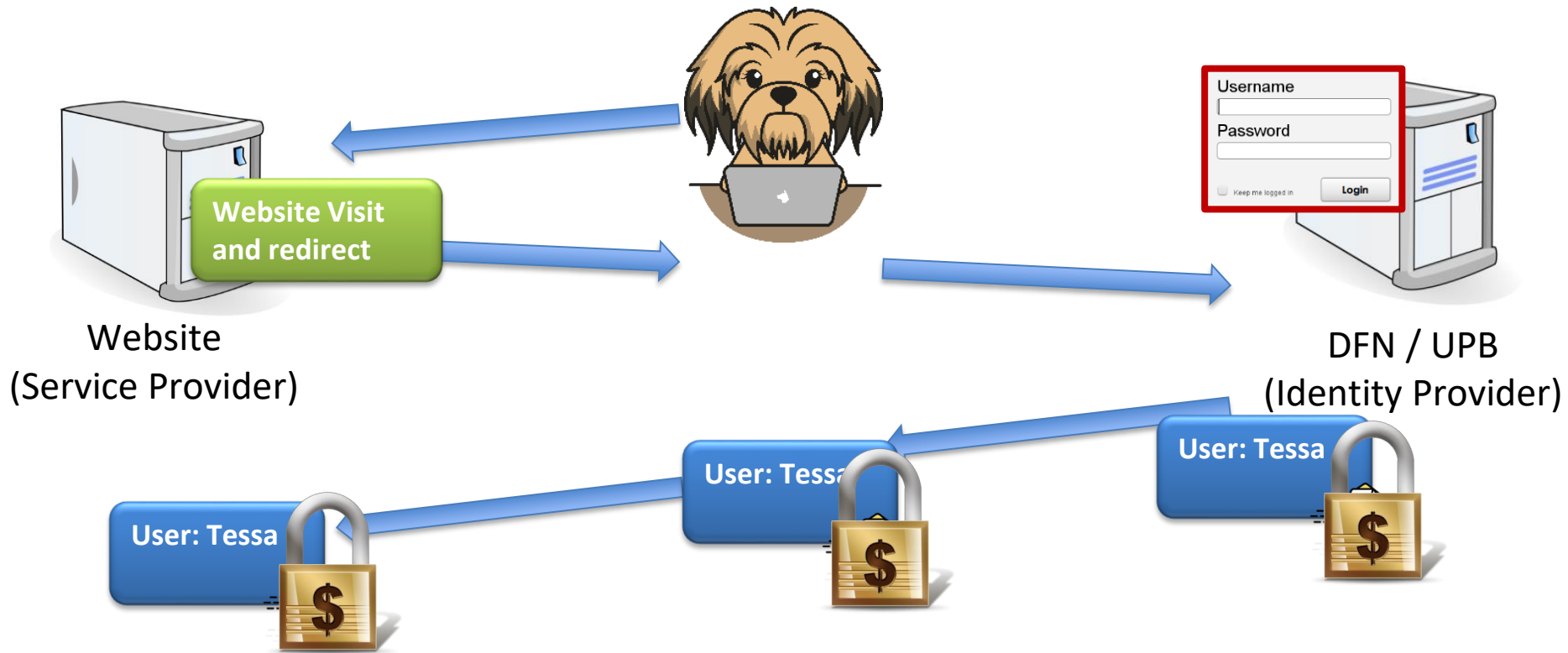
- Next week: External lecture by Vladislav Mladenov on Restful Web Services

Introduction to OAuth and OIDC

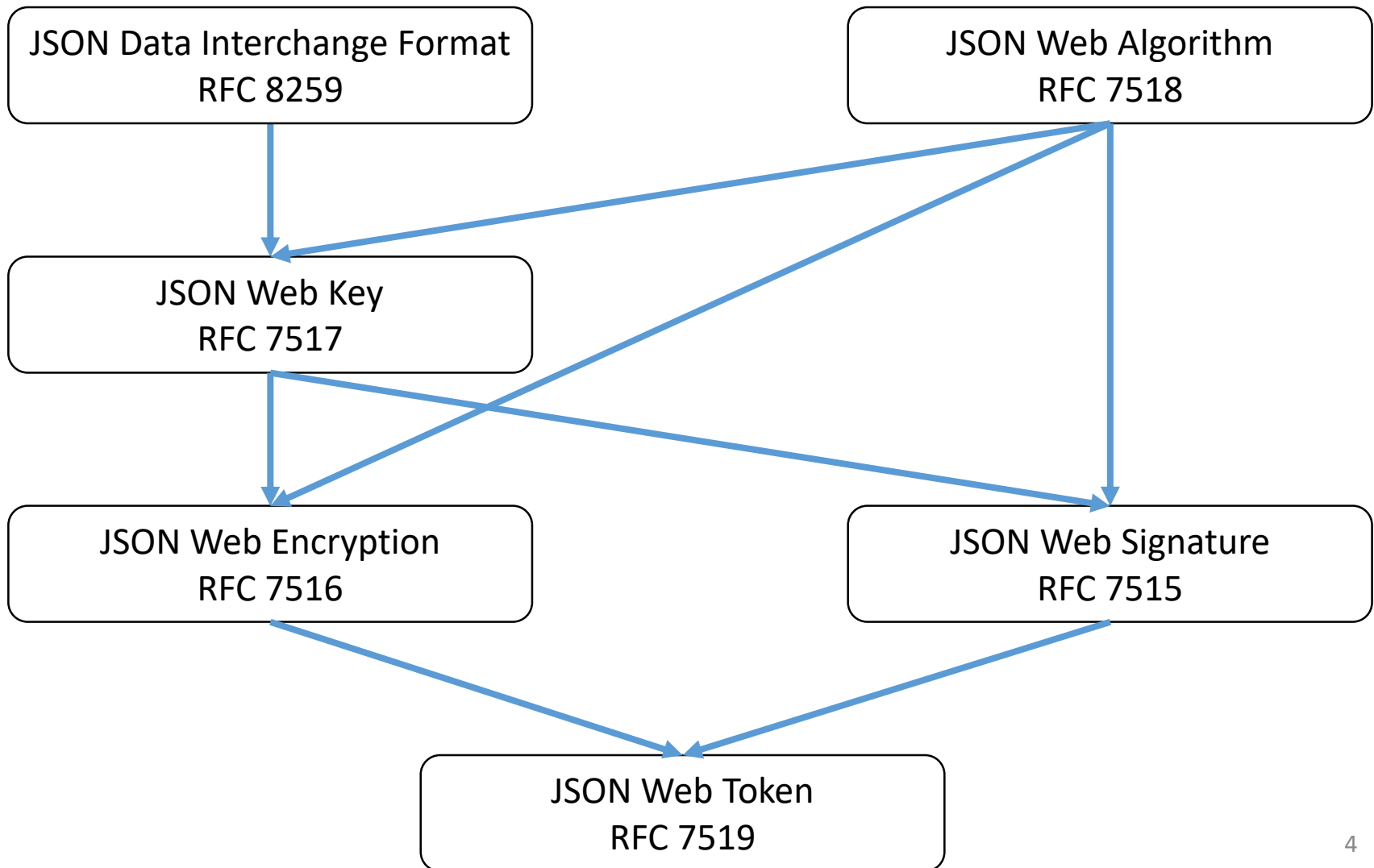
Web Security
Juraj Somorovsky

Summer Semester 2024
Paderborn University

SAML-based Single Sign-On



Overview: JavaScript Object Signing and Encryption (JOSE) working group



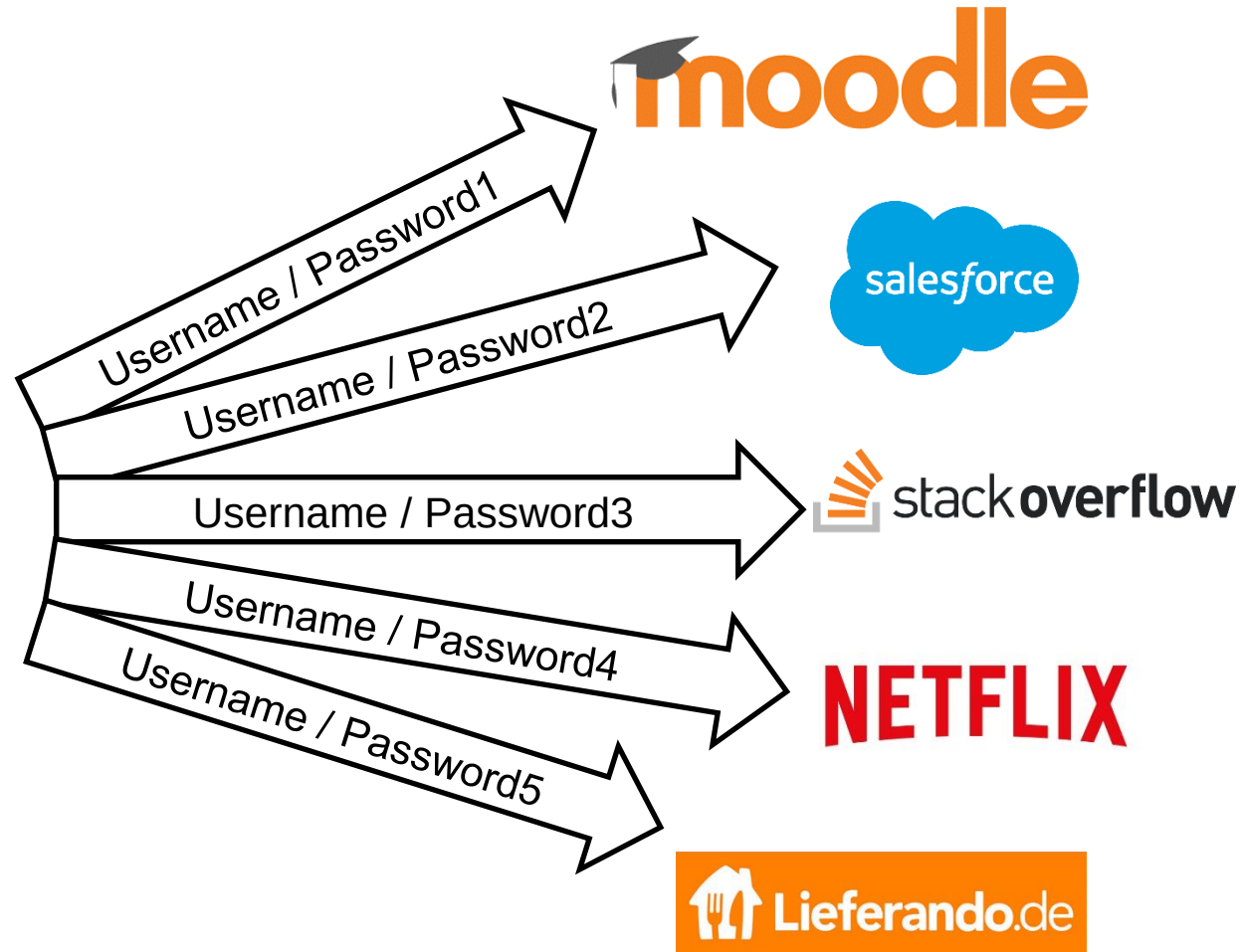
Overview

1. Web technologies
2. Web attacks
3. XML and Single Sign-On (SAML)
4. JSON and **OIDC / OAuth**
5. JSON Web Services

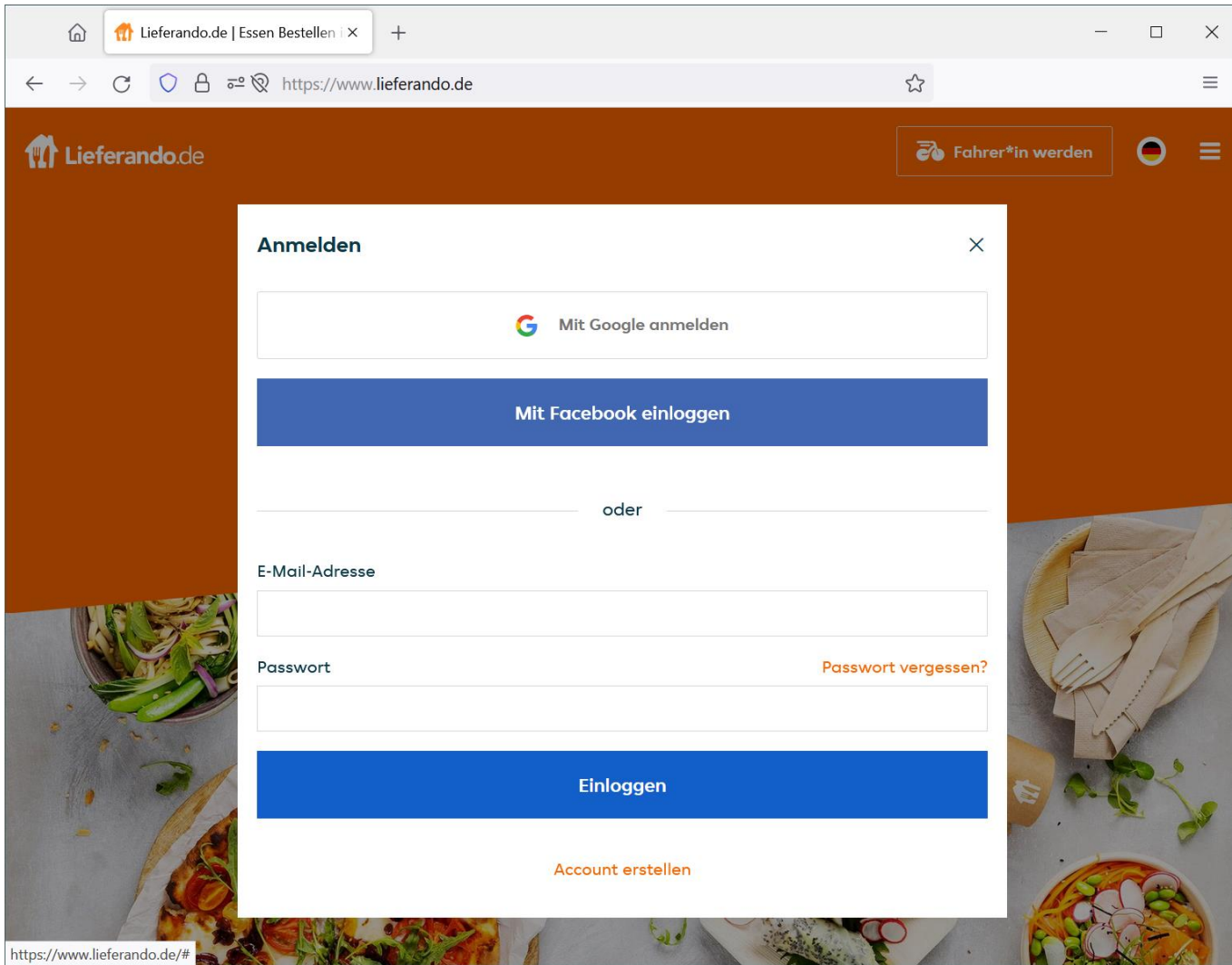
Overview

- 
1. Motivation
 2. Short History
 3. OAuth vs. OpenID Connect
 4. OpenID Connect Code Flow in Detail
 5. Flows
 6. Attacks
 1. Attacks on ID token
 2. Attacks on Other Parameters

The Password Dilemma



Single Sign-On Overview



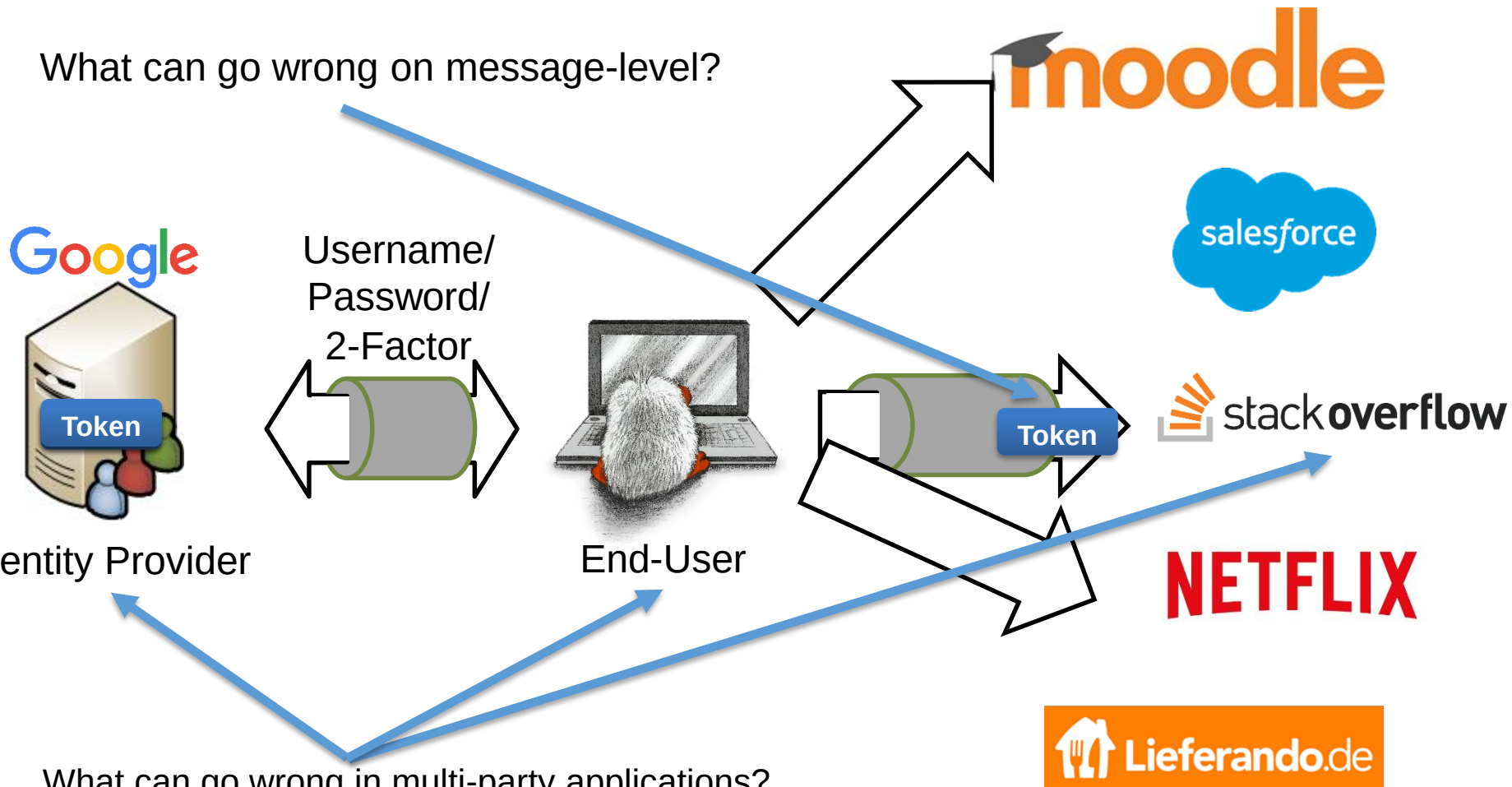
The screenshot shows a web browser window with the Lieferando.de website. The browser's address bar displays "https://www.lieferando.de". The website's header includes the Lieferando.de logo, a "Fahrer*in werden" button, and a German flag. A modal titled "Anmelden" is centered on the screen, featuring a close button (X) in the top right corner. The modal contains the following elements:

- A button labeled "Mit Google anmelden" with the Google logo.
- A blue button labeled "Mit Facebook einloggen".
- A separator line with the word "oder" in the center.
- An input field labeled "E-Mail-Adresse".
- An input field labeled "Passwort" with a link "Passwort vergessen?" to its right.
- A blue button labeled "Einloggen".
- A link "Account erstellen" at the bottom.

The background of the website shows a collage of food items, including a pizza, a salad, and a bowl of food.

The Single Sign-On Ecosystem

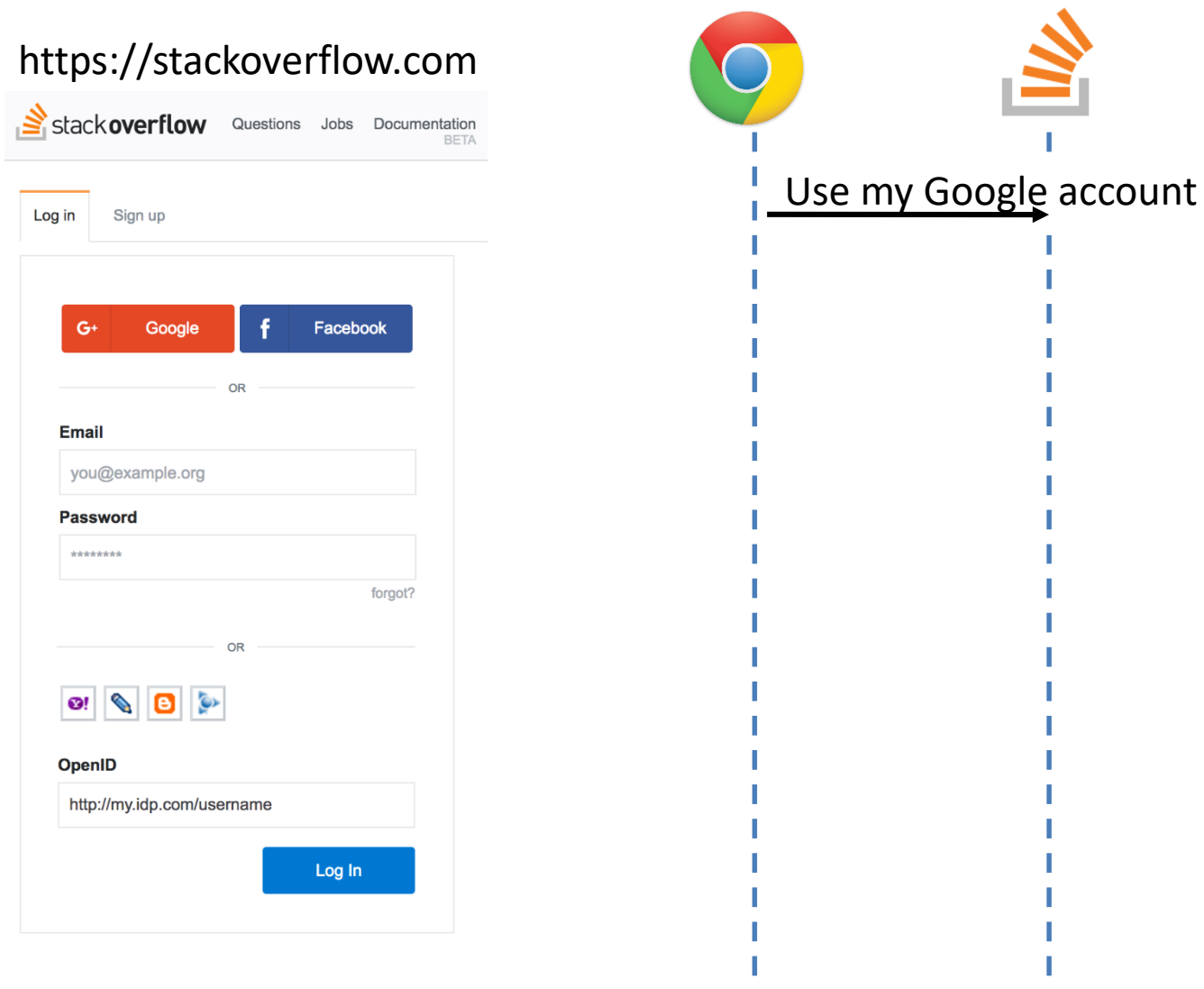
What can go wrong on message-level?



Single Sign-On



Login at StackOverflow with SSO



Login at StackOverflow with SSO

<https://accounts.google.com>



One account. All of Google.

Sign in with your Google Account

Christian.Mainka@gmail.com

Next

[Find my account](#)

[Create account](#)



Use my Google account

HTTP Redirect

Enter Google username

Login at StackOverflow with SSO

<https://accounts.google.com>



One account. All of Google.

Sign in with your Google Account

←

christian.mainka@gmail.com

Password

Sign in

☒ Stay signed in [Forgot password?](#)

[Sign in with a different account](#)



Use my Google account

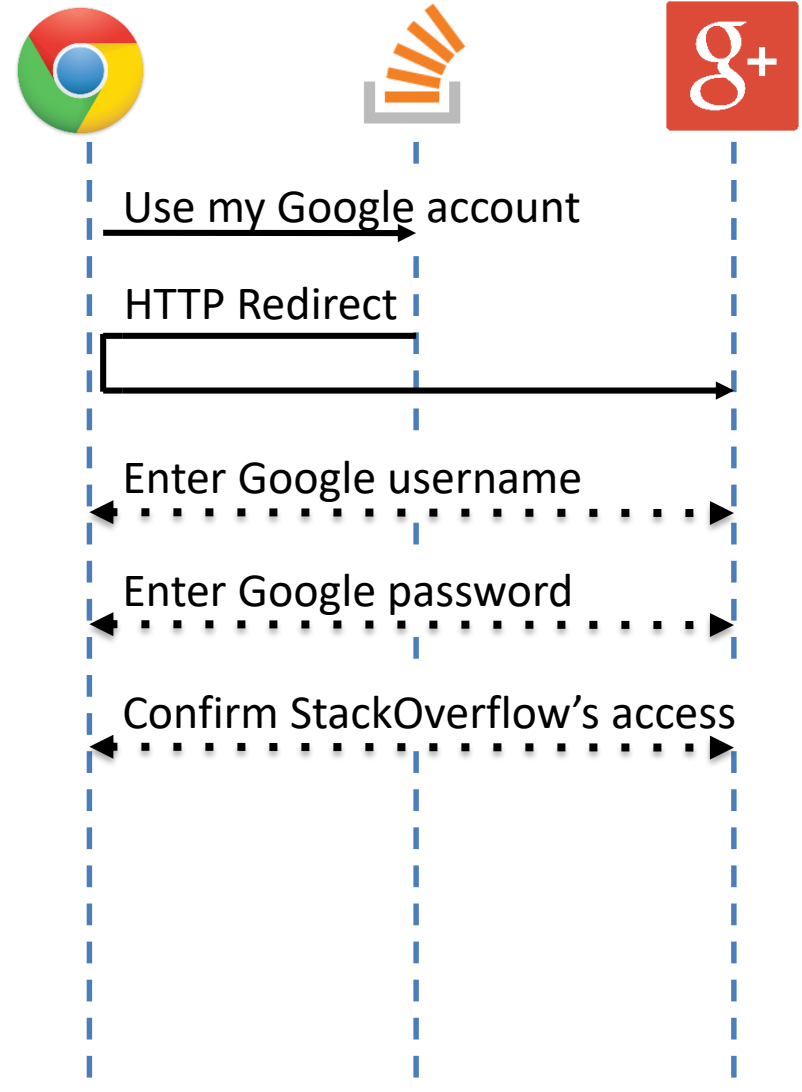
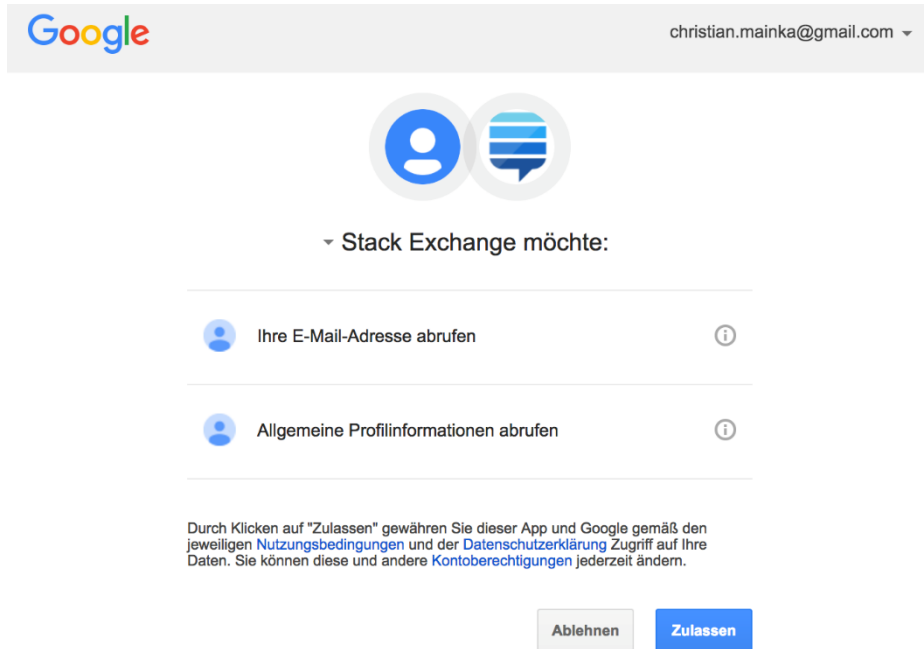
HTTP Redirect

Enter Google username

Enter Google password

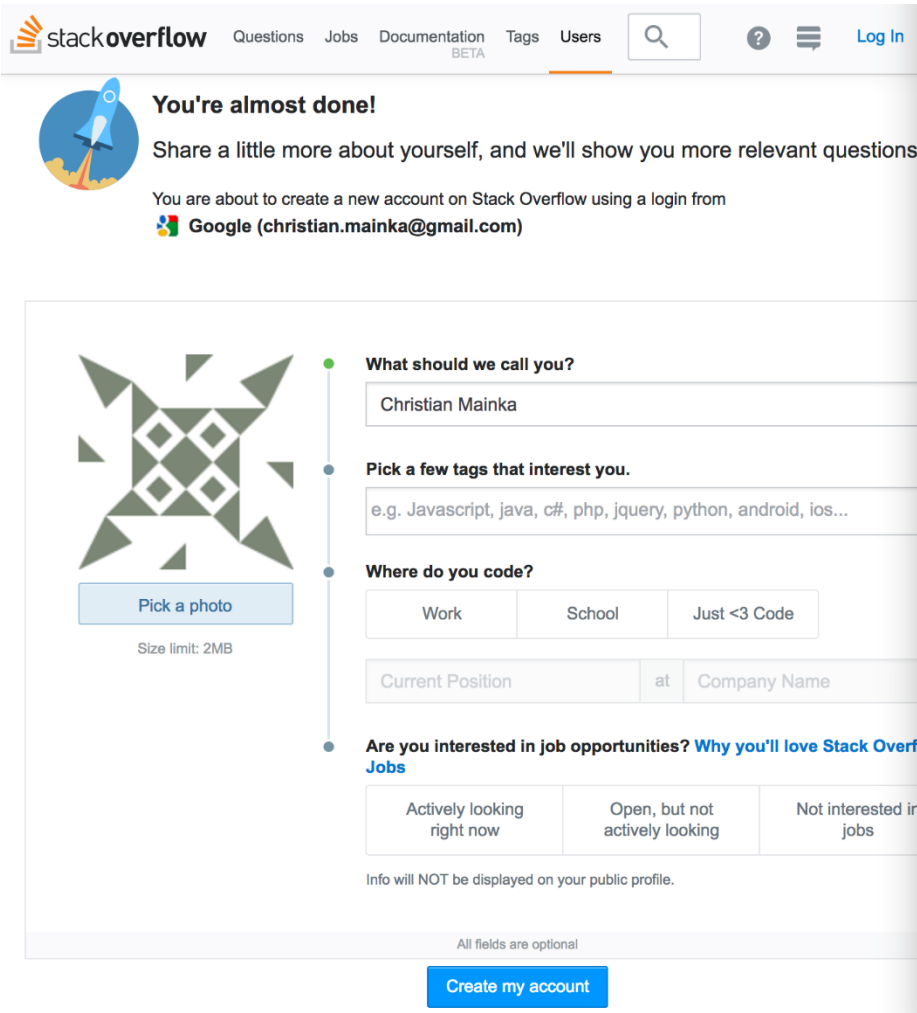
Login at StackOverflow with SSO

<https://accounts.google.com>

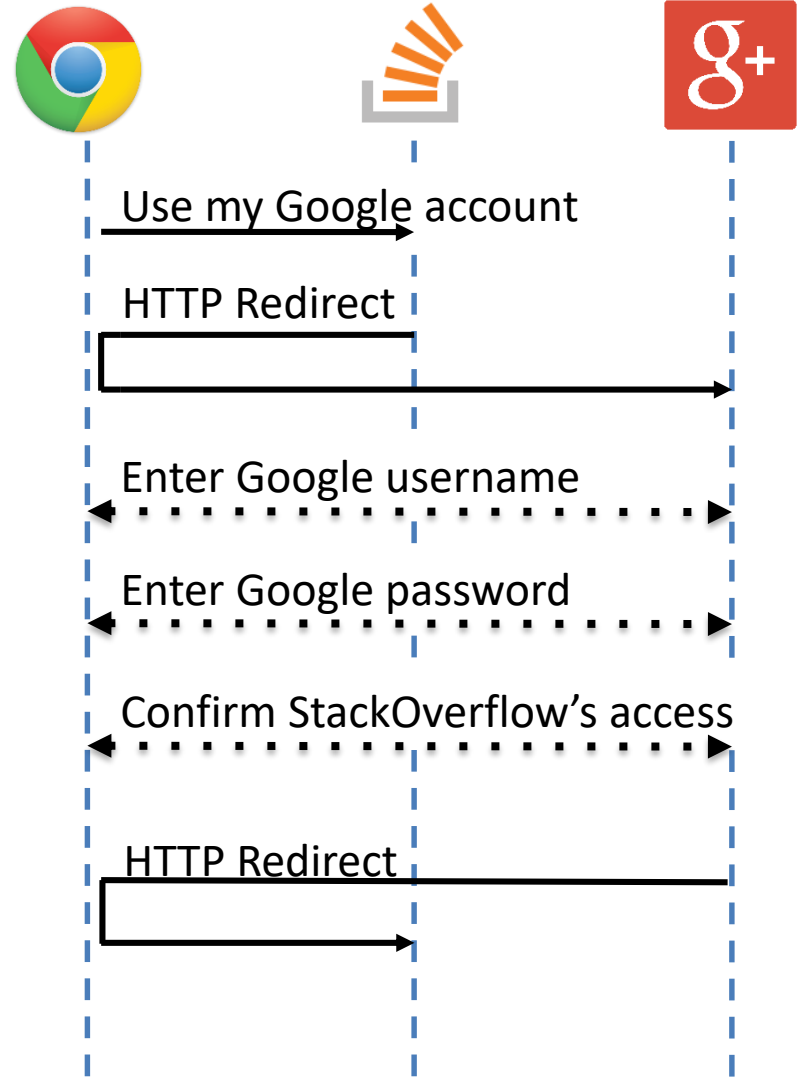


Login at StackOverflow with SSO

https://stackoverflow.com



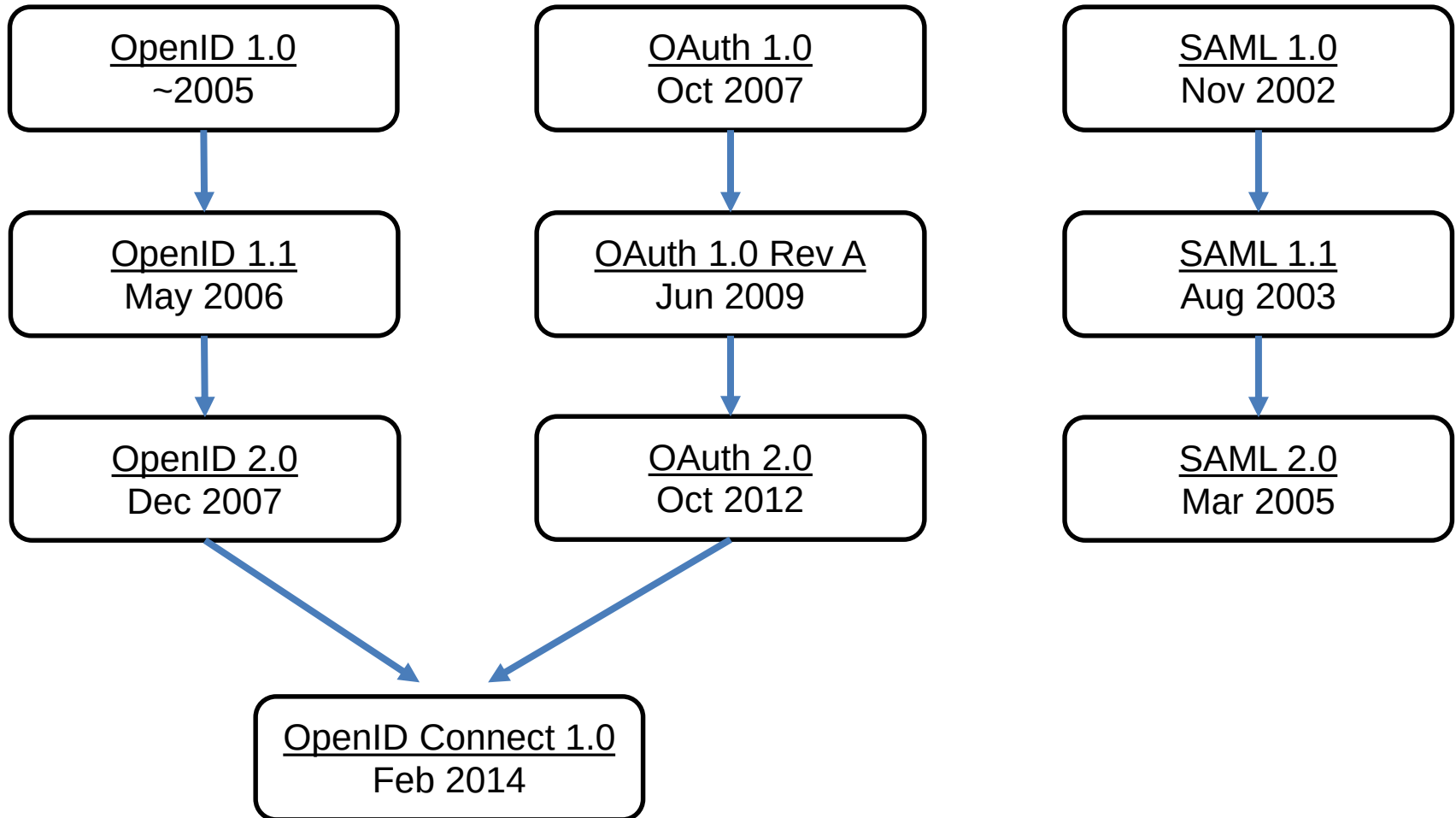
The screenshot shows the Stack Overflow login page. At the top, the URL is https://stackoverflow.com. The navigation bar includes links for Questions, Jobs, Documentation, Tags, Users, and a search bar. Below the navigation bar, a message says "You're almost done!" and "Share a little more about yourself, and we'll show you more relevant questions". It also states "You are about to create a new account on Stack Overflow using a login from Google (christian.mainka@gmail.com)". The main form has several sections: a profile picture upload area with a "Pick a photo" button and a "Size limit: 2MB" note; a section titled "What should we call you?" with a text input field containing "Christian Mainka"; a section titled "Pick a few tags that interest you." with a text input field containing "e.g. Javascript, java, c#, php, jquery, python, android, ios..."; a section titled "Where do you code?" with three radio buttons labeled "Work", "School", and "Just <3 Code"; a section for "Current Position" and "Company Name"; and a section titled "Are you interested in job opportunities? Why you'll love Stack Overflow Jobs" with three radio buttons labeled "Actively looking right now", "Open, but not actively looking", and "Not interested in jobs". At the bottom, there is a "Create my account" button.



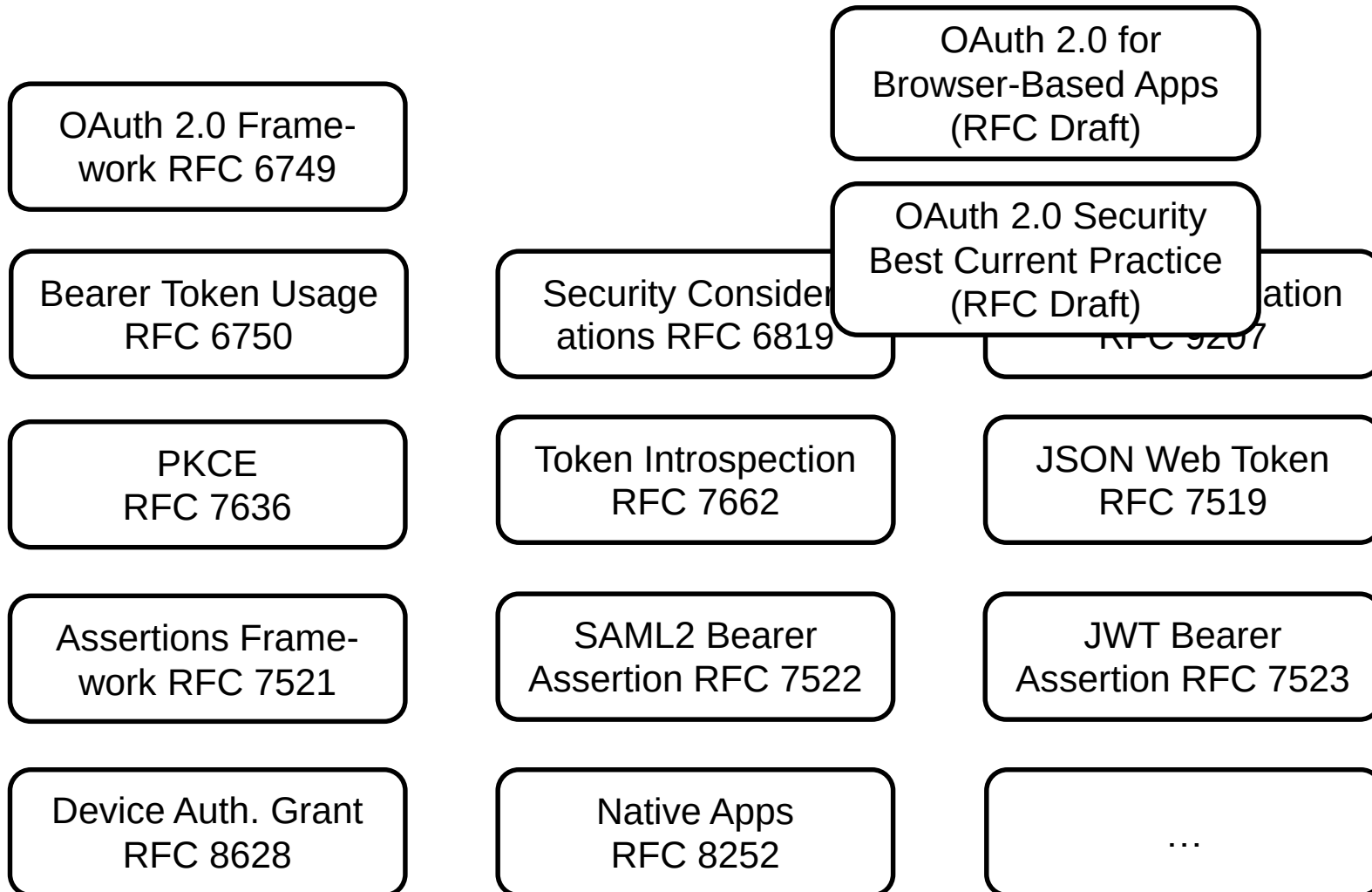
Overview

1. Motivation
-  2. Short History
3. OAuth vs. OpenID Connect
4. OpenID Connect Code Flow in Detail
5. Flows
6. Attacks
 1. Attacks on ID token
 2. Attacks on Other Parameters

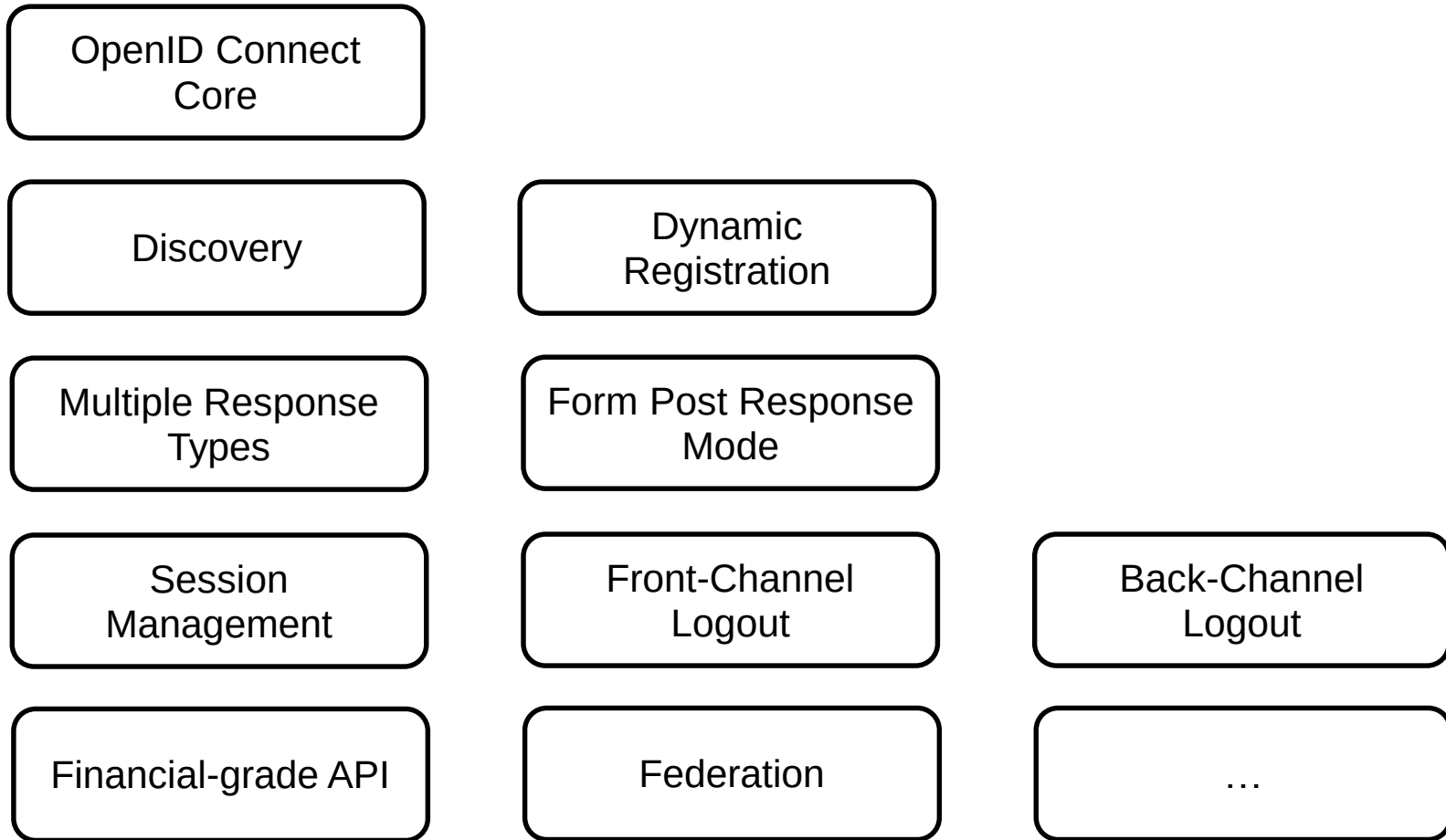
History of Single Sign-On



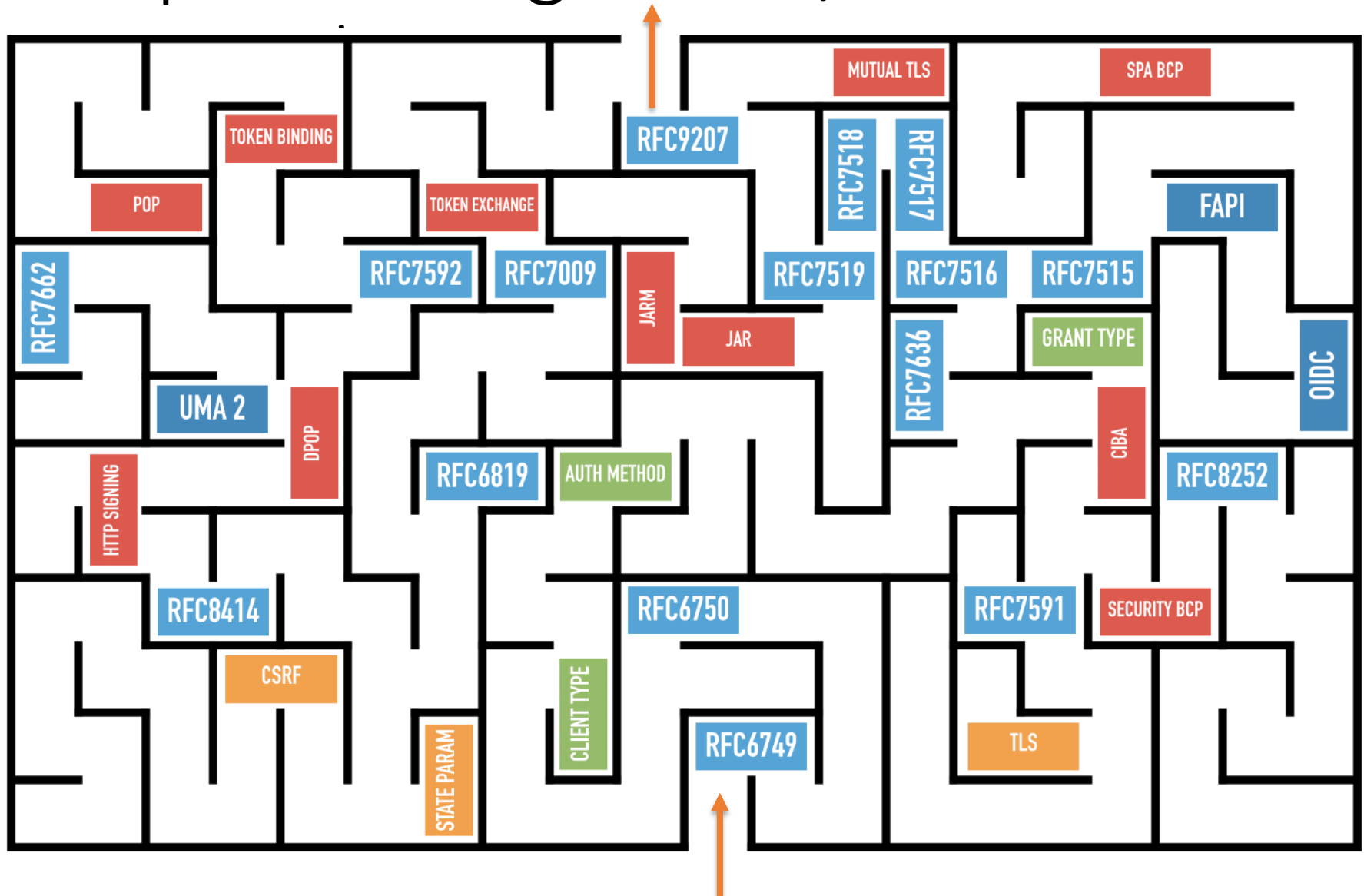
OAuth (IETF)



OpenID Connect (OpenID Foundation)



Implementing OAuth/OIDC



Overview

1. Motivation
2. Short History
-  3. OAuth vs. OpenID Connect
4. OpenID Connect Code Flow in Detail
5. Flows
6. Attacks
 1. Attacks on ID token
 2. Attacks on Other Parameters

OAuth 2.0 vs. OpenID Connect 1.0

OAuth 2.0



Client
Application

OAuth 2.0 vs. OpenID Connect 1.0

OpenID Connect 1.0



Client
Application

Why OAuth?

(=Delegated Authorization)



Consent-Page

Request for Permission

Social Me is requesting permission to do the following:

**Access my basic information**
Includes name, profile picture, gender, networks, user ID, list of friends, and any other information I've shared with everyone.

**Social Me**

**Access my data any time**
Social Me may access my data when I'm not using the application

**Access my profile Information**
About Me, Activities, Interests, Groups, Events, Notes, Hometown, Current City, Education History and Work History

**Access my photos and videos**
Photos Uploaded by Me, Videos Uploaded by Me and Photos and Videos of Me

**Access my friends' Information**
Hometowns, Current Cities, Activities, Interests, Education History, Work History, Groups, Events, Notes, Photos, Videos, Photos and Videos of Them and 'About Me' Details

[Report Application](#)

Logged in as Christopher Peplin (Not You?)

Allow

Don't Allow

3G 11:15 AM

f Facebook

Don't Allow

Permissions

AndroidSSO is requesting optional permissions:

**Access my data any time**
This app may access my data when I'm not using the application

**Check-ins**
This app may read my check-ins, friends' check-ins and publish check-ins on my behalf.

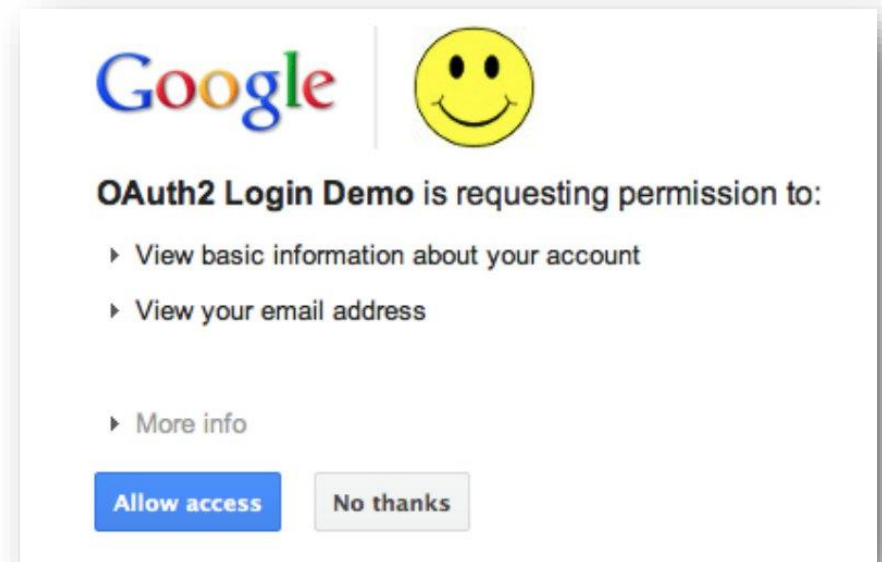
Allow All

Allow None

25

OAuth is nice if...

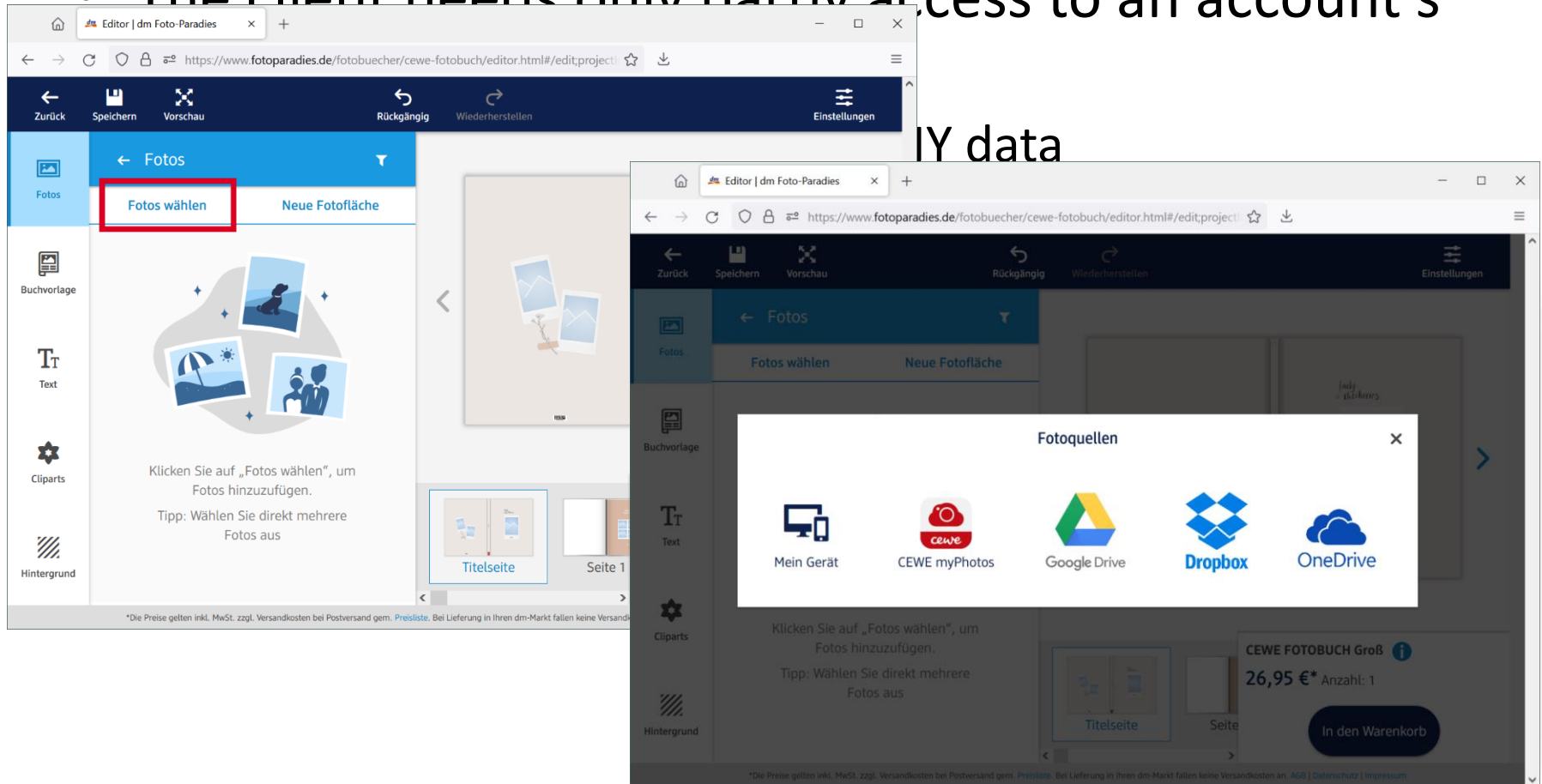
- The client needs only partly access to an account's data
 - Your Google account holds MANY data
 - Allow your calendar app only access to necessary data
 - “Older” apps require End-User's password
 - Bad!
 - Password
 - = Highest credential
 - = Game over



OAuth is nice if...

- The client needs only partly access to an account's

data



Why OpenID Connect? (=Delegated Authentication)

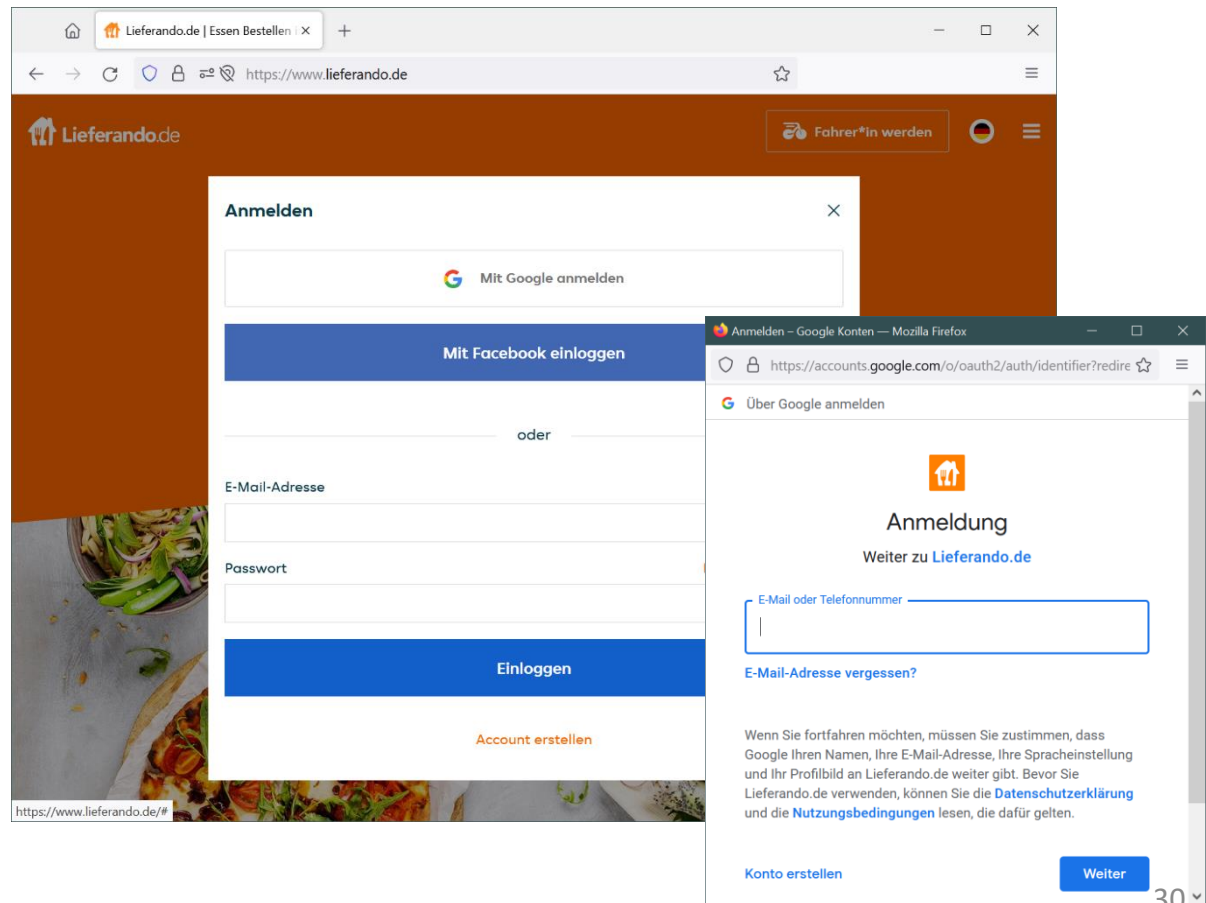


OpenID Connect is nice if...

- The Client needs to know WHO the End-User is
 - Contract
 - Voting
 - Shopping

OpenID Connect is nice if...

- The Client needs to know WHO the End-User is
 - Contract
 - Voting
 - Shopping



OAuth is designed for:

- Delegated Authorization

OAuth is **not** designed for:

- Authentication

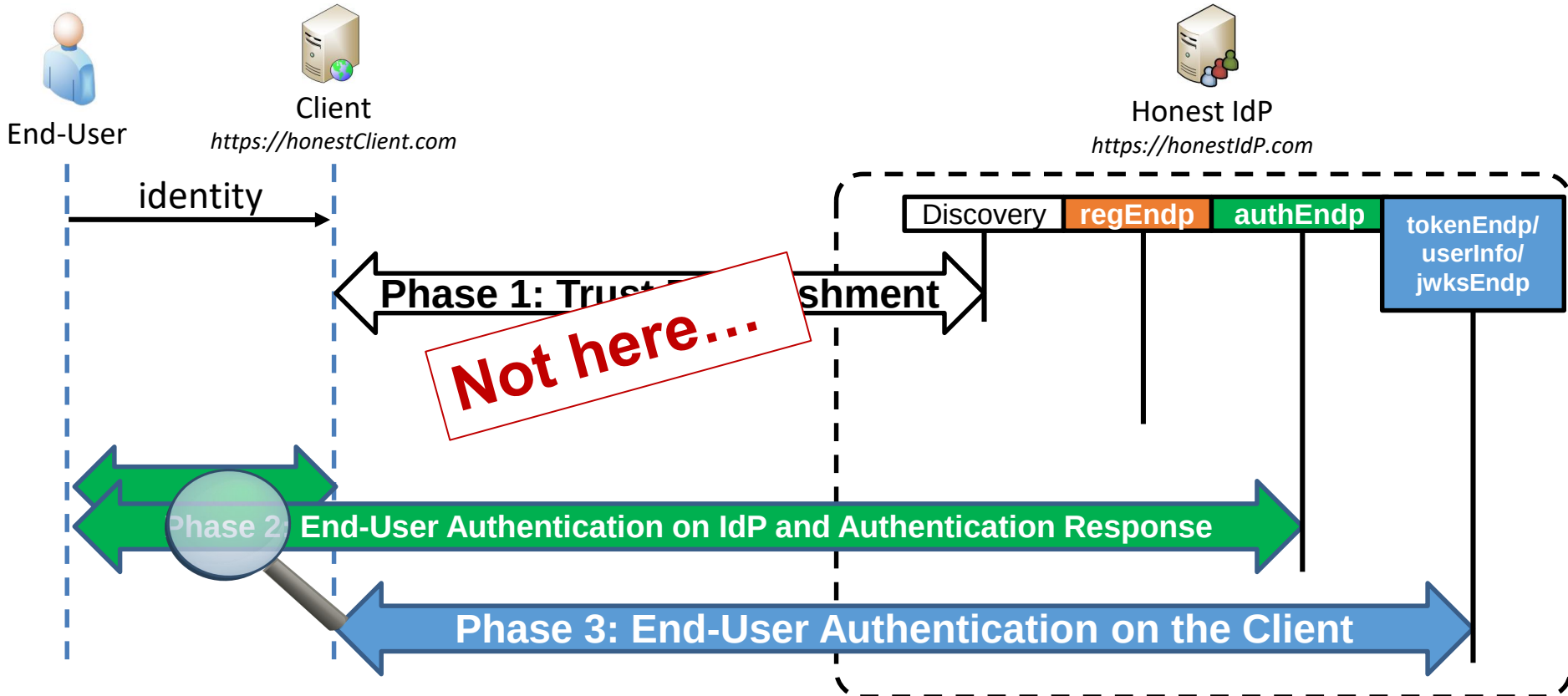
Overview

1. Motivation
2. Short History
3. OAuth vs. OpenID Connect
-  4. OpenID Connect Code Flow in Detail
5. Flows
6. Attacks
 1. Attacks on ID token
 2. Attacks on Other Parameters

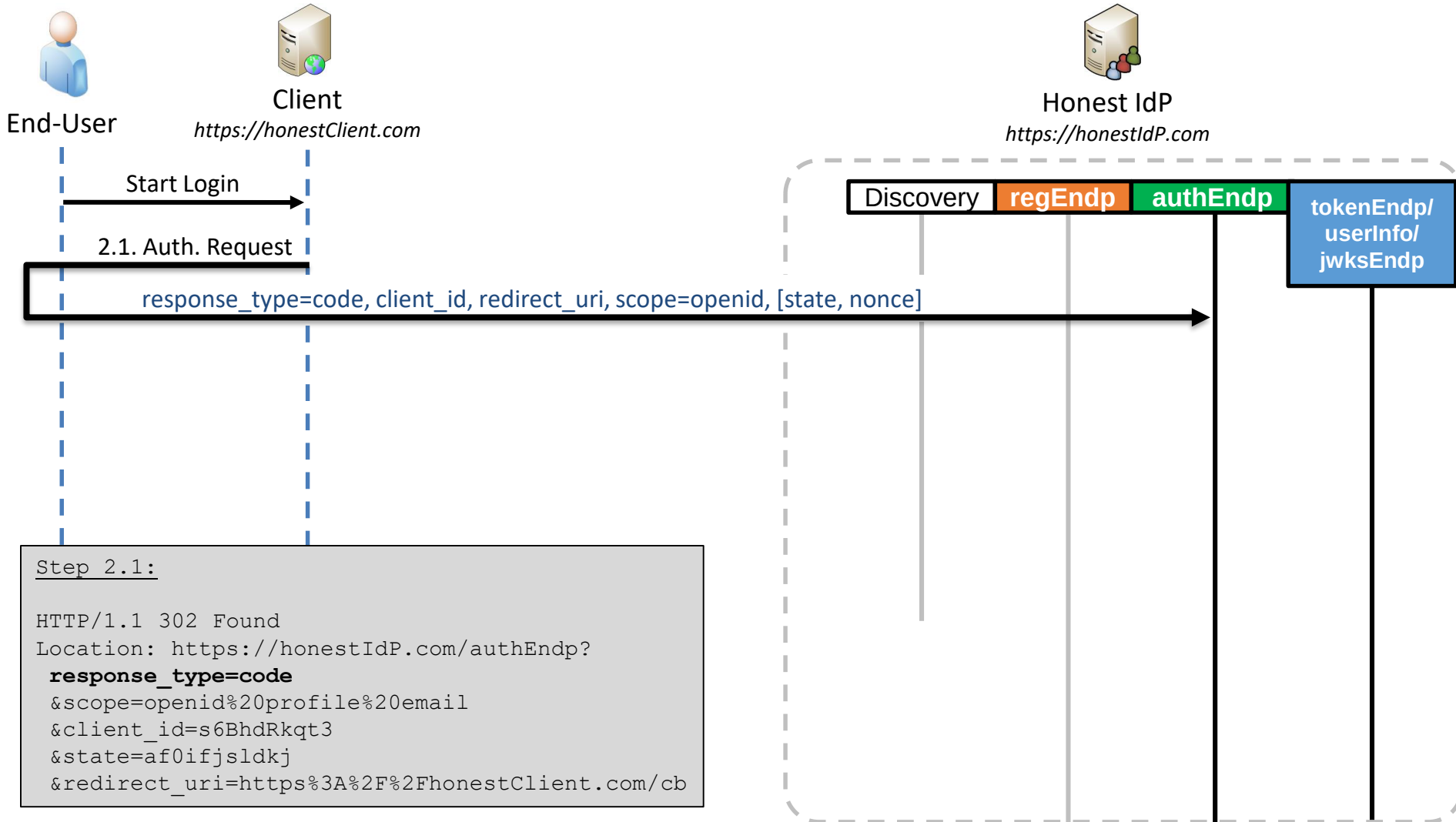
Notation

- Client = Service Provider
- (If necessary) We explicitly use:
 - End-User
 - User Agent (Browser)

OpenID Connect in Three Phases



OpenID Connect: End-User Authentication on IdP (Code Flow)



Authentication Request Details (1/2)

Step 2.1:

```
HTTP/1.1 302 Found
Location: https://honestIdP.com/authEndp?
  response_type=code
  &scope=openid%20profile%20email
  &client_id=s6BhdRkqt3
  &state=af0ifjsldkj
  &redirect_uri=https%3A%2F%2FhonestClient.com/cb
```



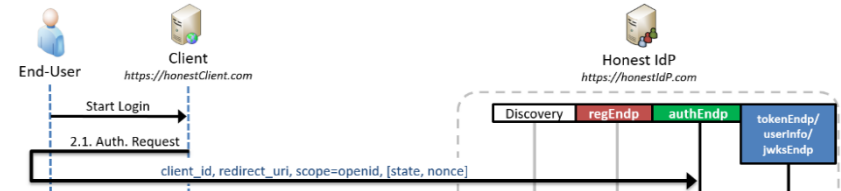
Three+ main flows

Flow	response_type=	
Code	code	• Authentication in back-channel
Implicit	id_token id_token token	• Authentication in front-channel
Hybrid	code id_token code token code id_token token	• Hybrid = Authentication and/or Authorization in both channels

Authentication Request Details (2/2)

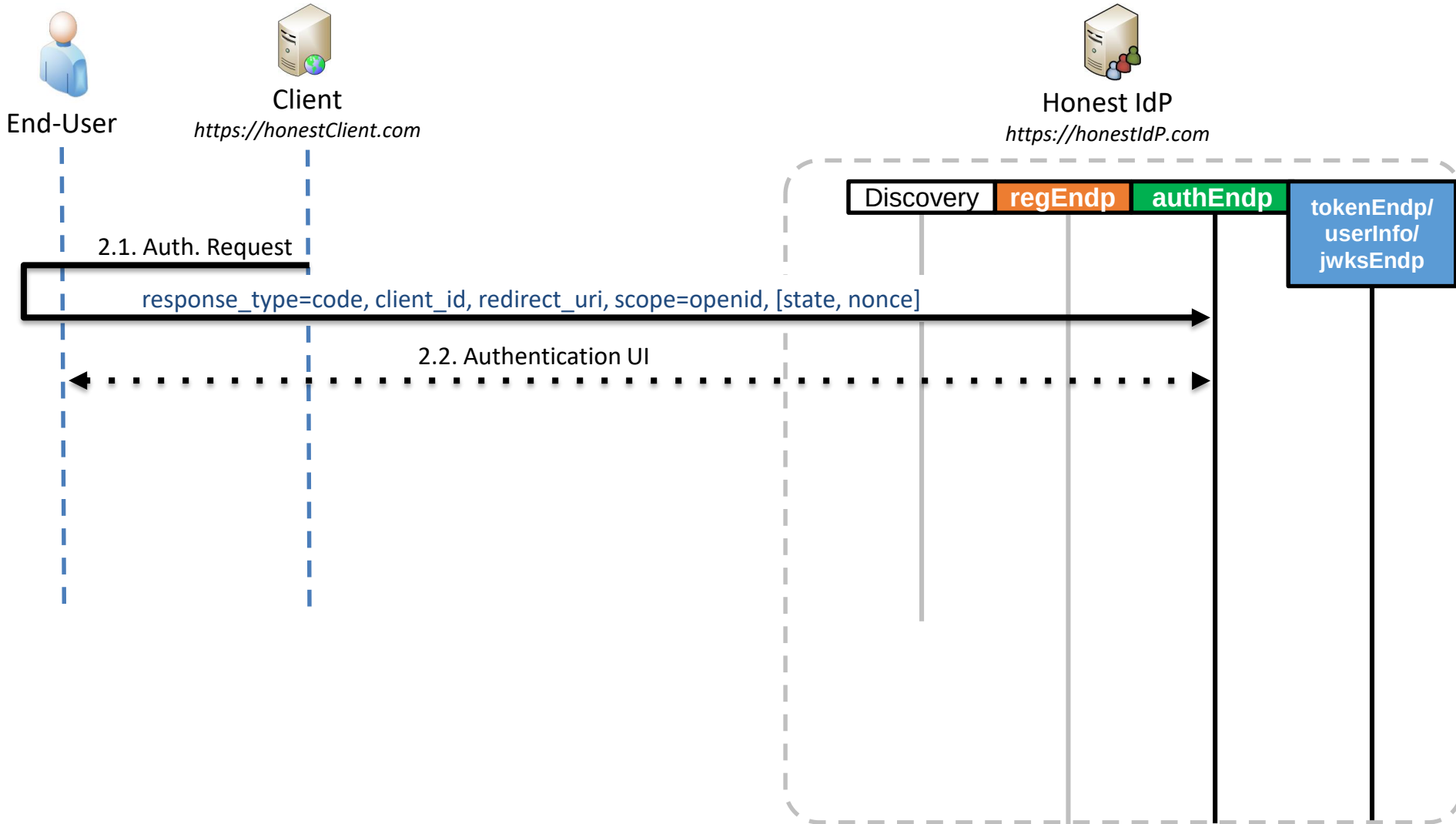
Step 2.1:

```
HTTP/1.1 302 Found
Location: https://honestIdP.com/authEndp?
  response_type=code
  &scope=openid%20profile%20email
  &client_id=s6BhdRkqt3
  &state=af0ifjsldkj
  &redirect_uri=https%3A%2F%2FhonestClient.com/cb
```



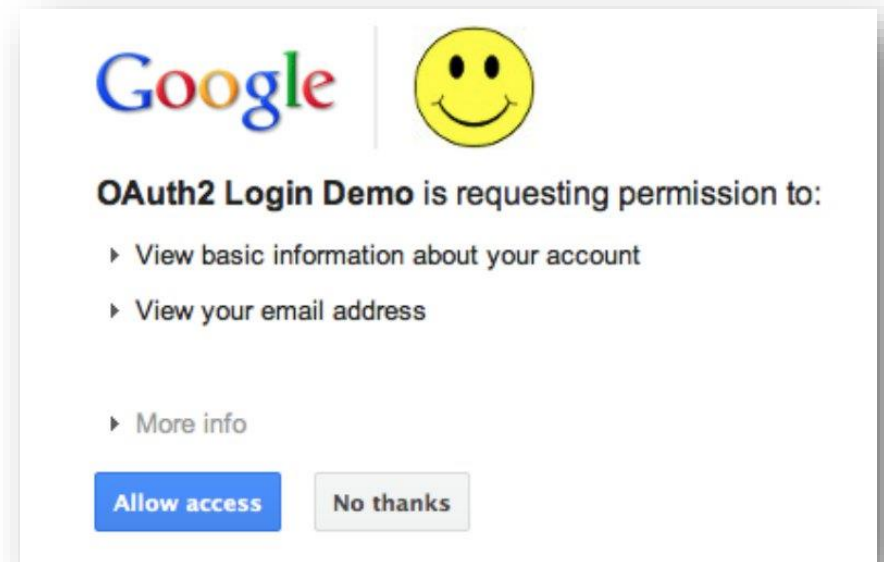
- *scope*: must contain *openid*, defines access
- *client_id*: identification of the Client
- *state* (recommended): CSRF protection
- *redirect_uri*: redirect End-User after login to this URI
- *nonce*: prevents replay attacks (+ CSRF protection)

OpenID Connect: End-User Authentication on IdP (Code Flow)



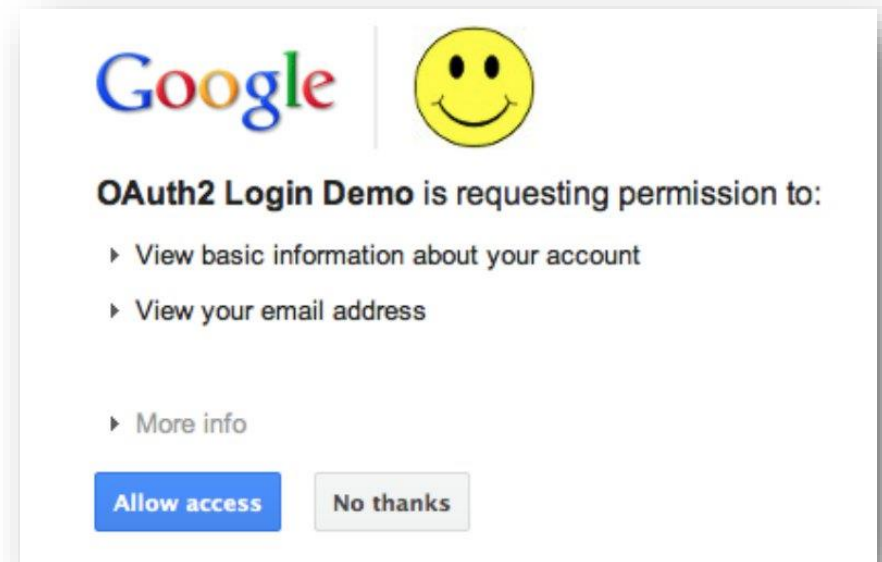
The Consent-Page

- Why should you use it?
- Which information to show?
- What can go wrong?

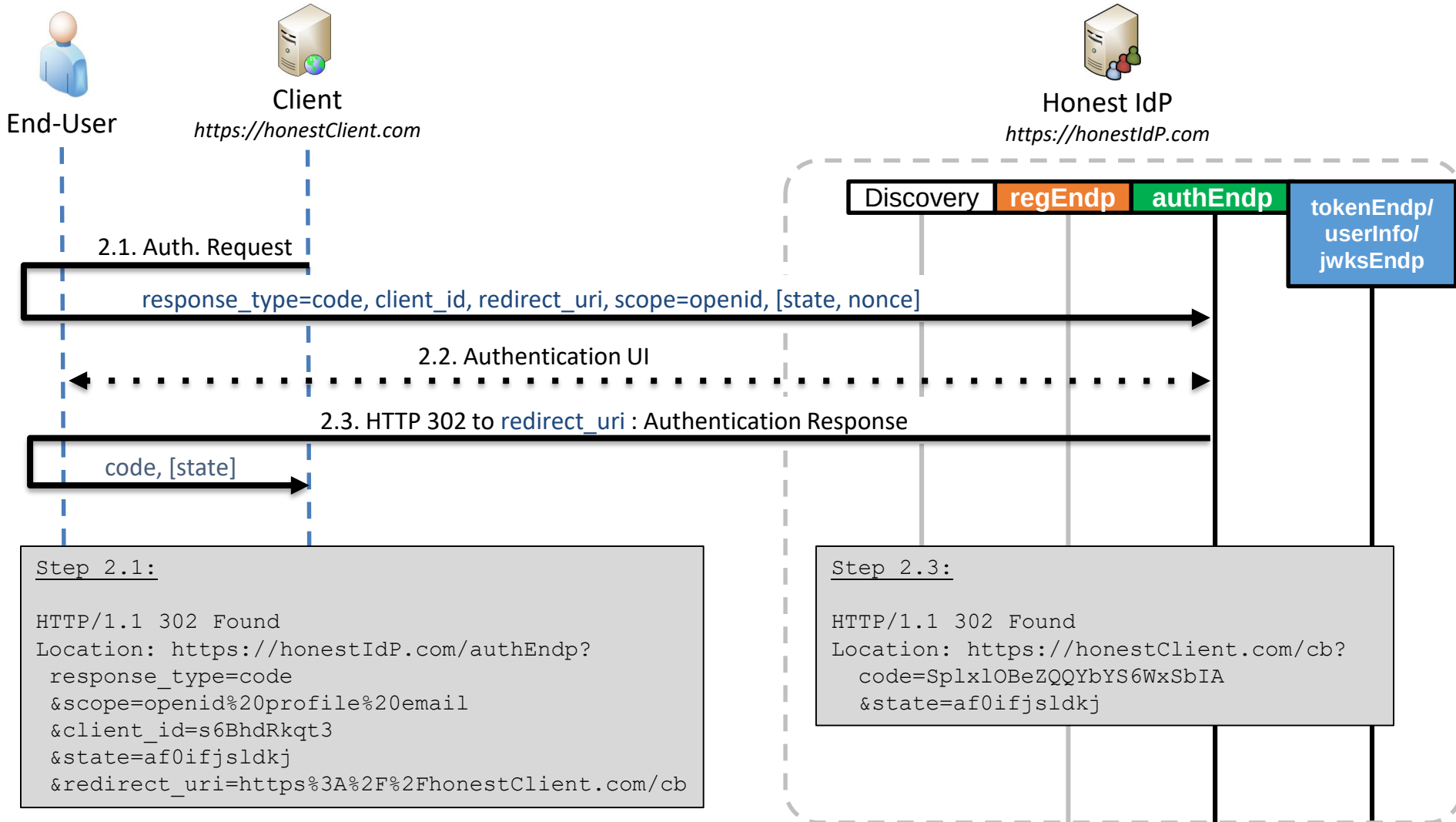


The Consent-Page

- Possible in OAuth and OpenID Connect
 - But not specified!
- Protects from “invisible logins”
 - so-called CSRF Attacks
- Transparency to user
 - Who wants access?
 - > client
 - Access to what?
 - > scopes



OpenID Connect: End-User Authentication on IdP (Code Flow)

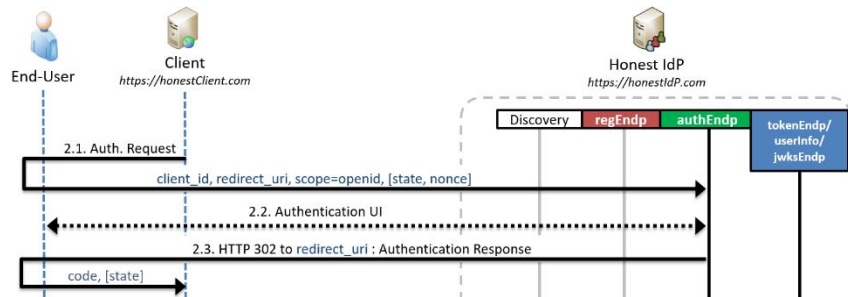


Authentication Response Details

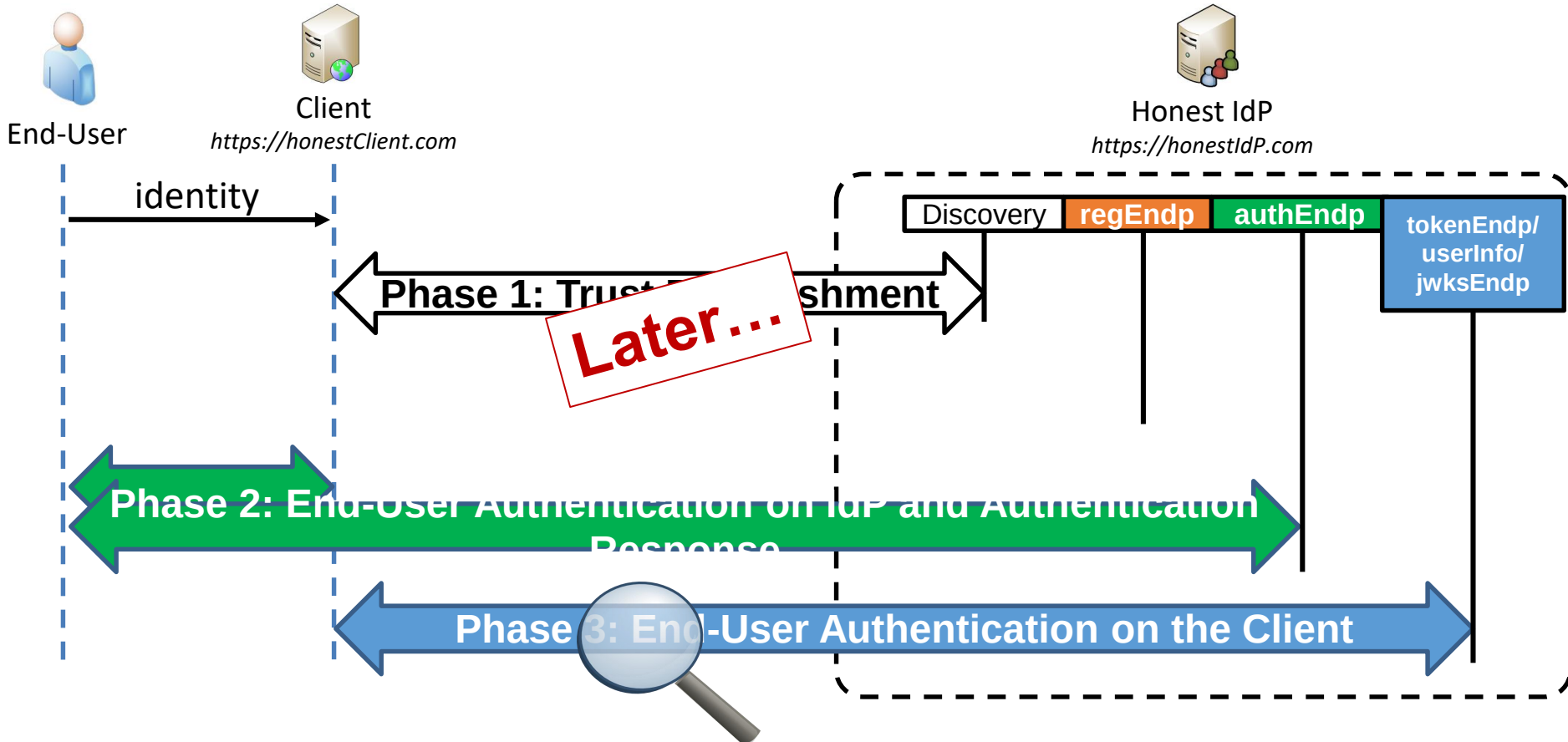
Step 2.3:

```
HTTP/1.1 302 Found
Location: https://honestClient.com/cb?
  code=Sp1xl0BeZQQYbYS6WxSbIA
  &state=af0ifjsldkj
```

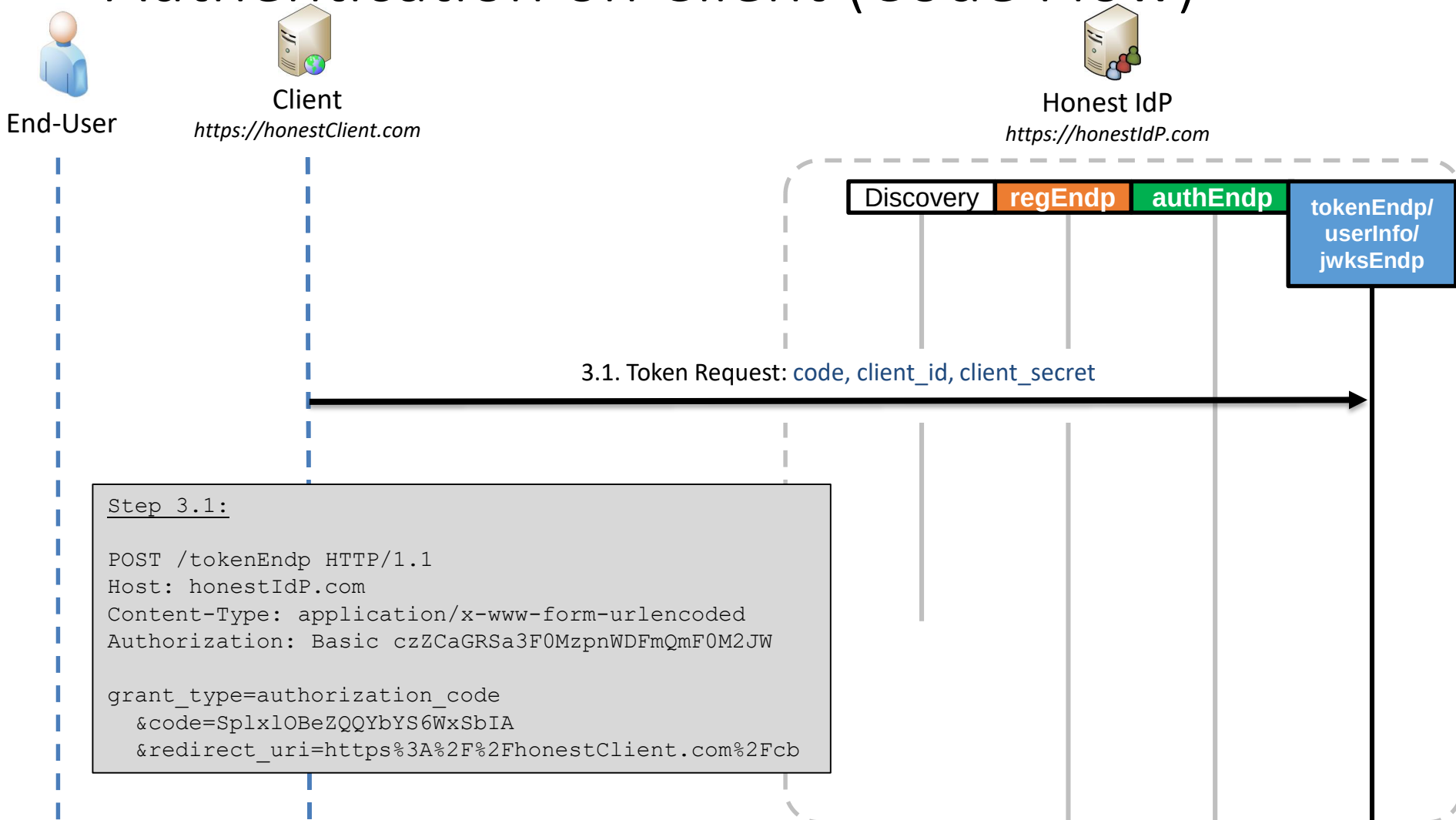
- Location:
 - Comes from *redirect_uri*
 - Defined in previous request
- *code*:
 - Reference for Tokens
 - Remember: `response_type=code`
- *state*: CSRF protection



OpenID Connect in Three Phases



OpenID Connect: End-User Authentication on Client (Code Flow)

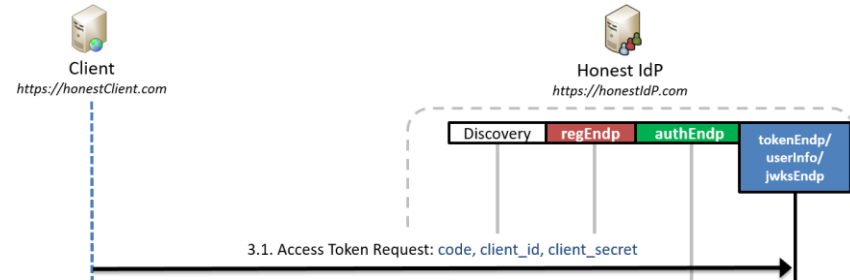


Token Request

Step 3.1:

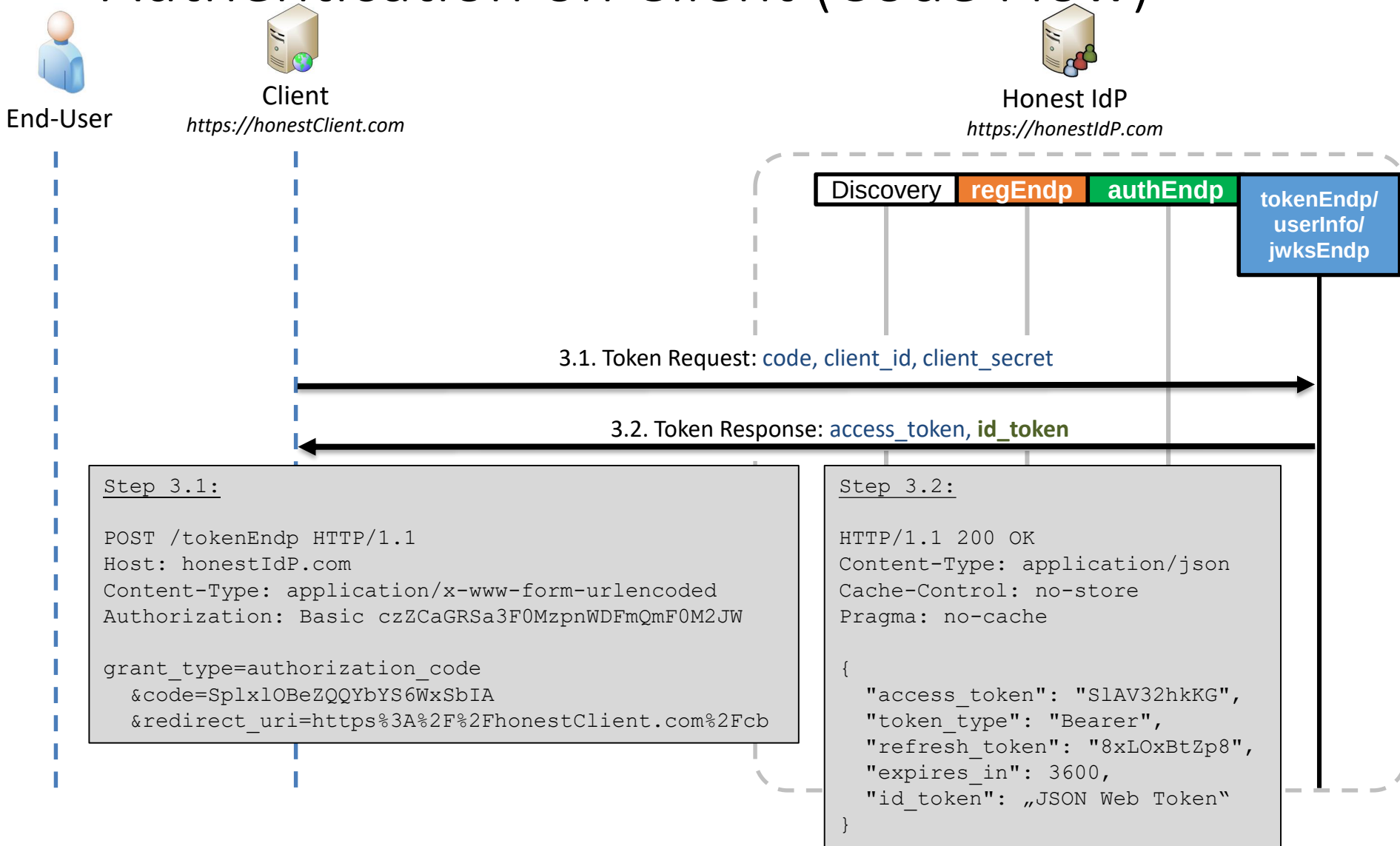
```
POST /tokenEndp HTTP/1.1
Host: honestIdP.com
Content-Type: application/x-www-form-urlencoded
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW

grant_type=authorization_code
&code=Sp1x10BeZQQYbYS6WxSbIA
&redirect_uri=https%3A%2F%2FhonestClient.com%2Fcb
```



- Authorization
 - BasicAuth: `client_id/client_secret`
 - POST/GET also possible
- `grant_type`: From Authentication Request
- `code`: Reference to `id_token/access_token`
- `redirect_uri`: From Authentication Request

OpenID Connect: End-User Authentication on Client (Code Flow)

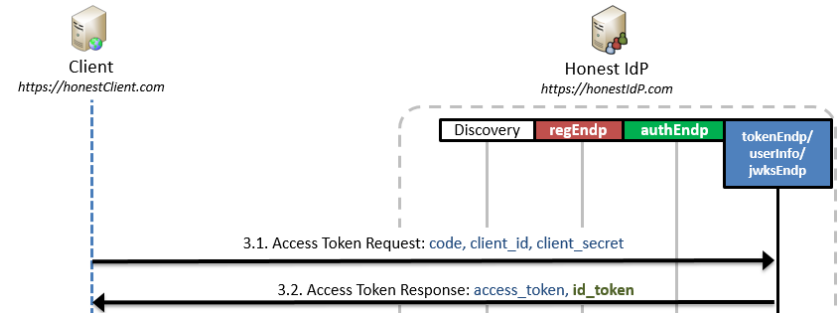


Token Response

Step 3.2:

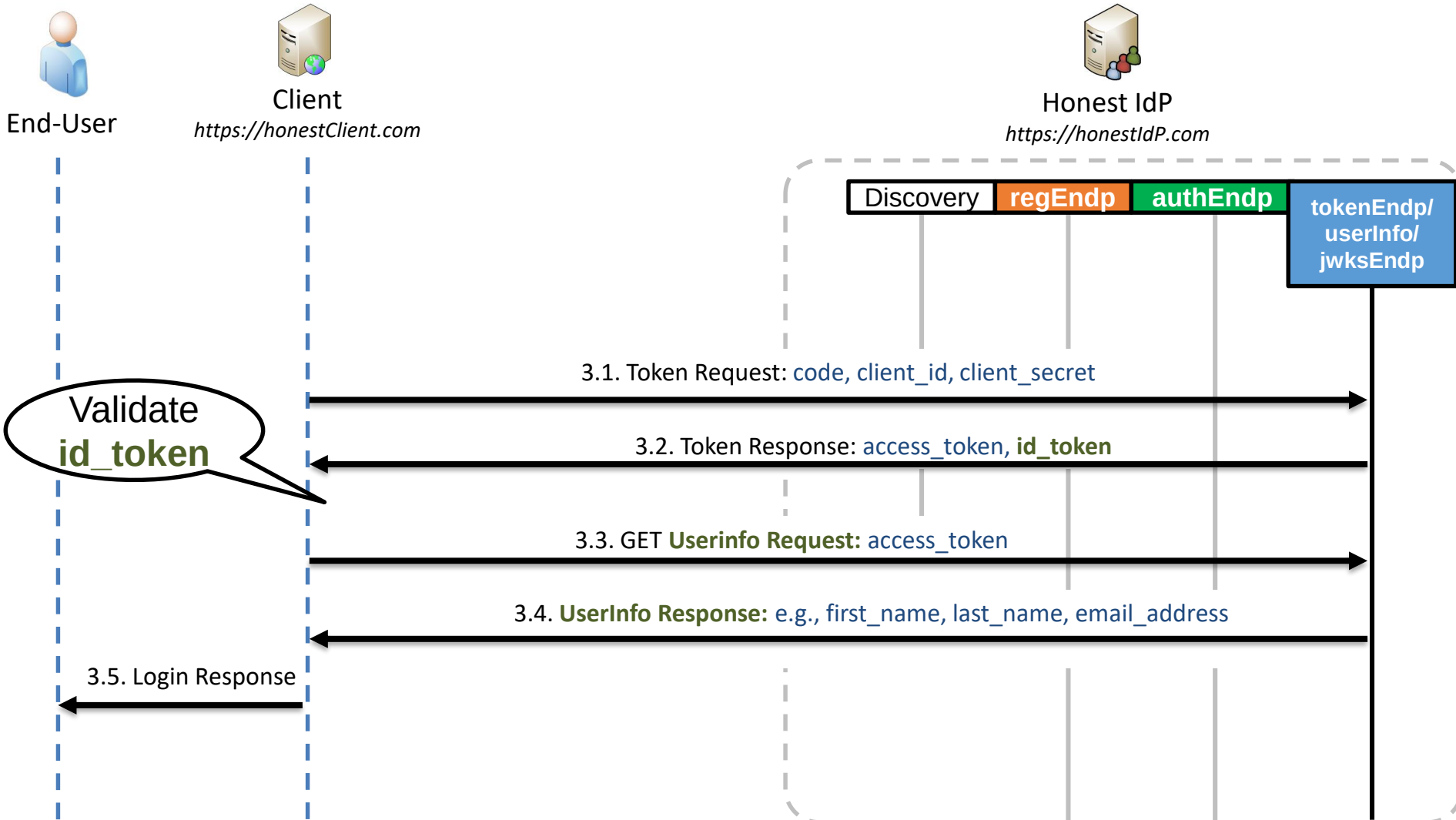
```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store
Pragma: no-cache
```

```
{
  "access_token": "SlAV32hkKG",
  "token_type": "Bearer",
  "refresh_token": "8xLOxBtZp8",
  "expires_in": 3600,
  "id_token": „JSON Web Token“
}
```

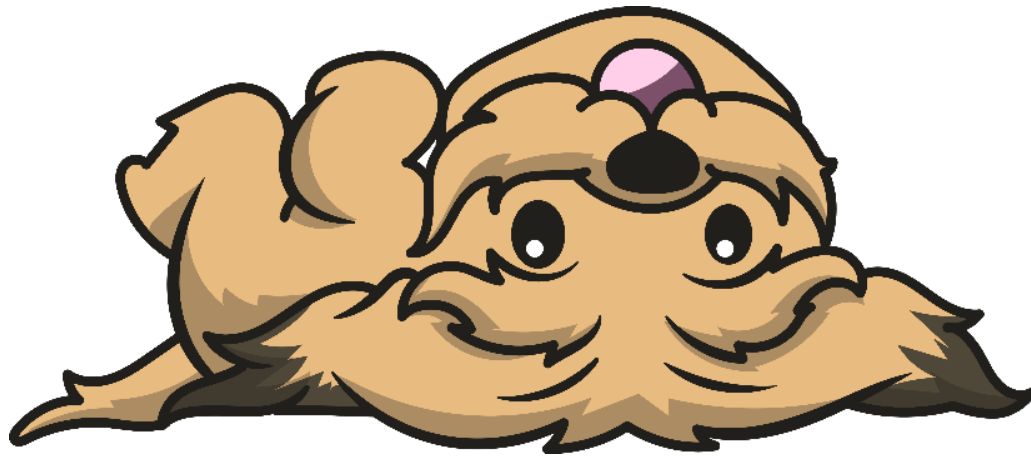


- Different Tokens can be returned
 - access_token/refresh_token -> OAuth
 - id_token -> OpenID Connect
 - Described in detail later

OpenID Connect: End-User Authentication on IdP (Code Flow)



Questions



V-Krit Evaluation (15 min)

- Juraj Somorovsky: K454P
- Exercise: RQ7ZW
- Link: <https://go.upb.de/vkrit>

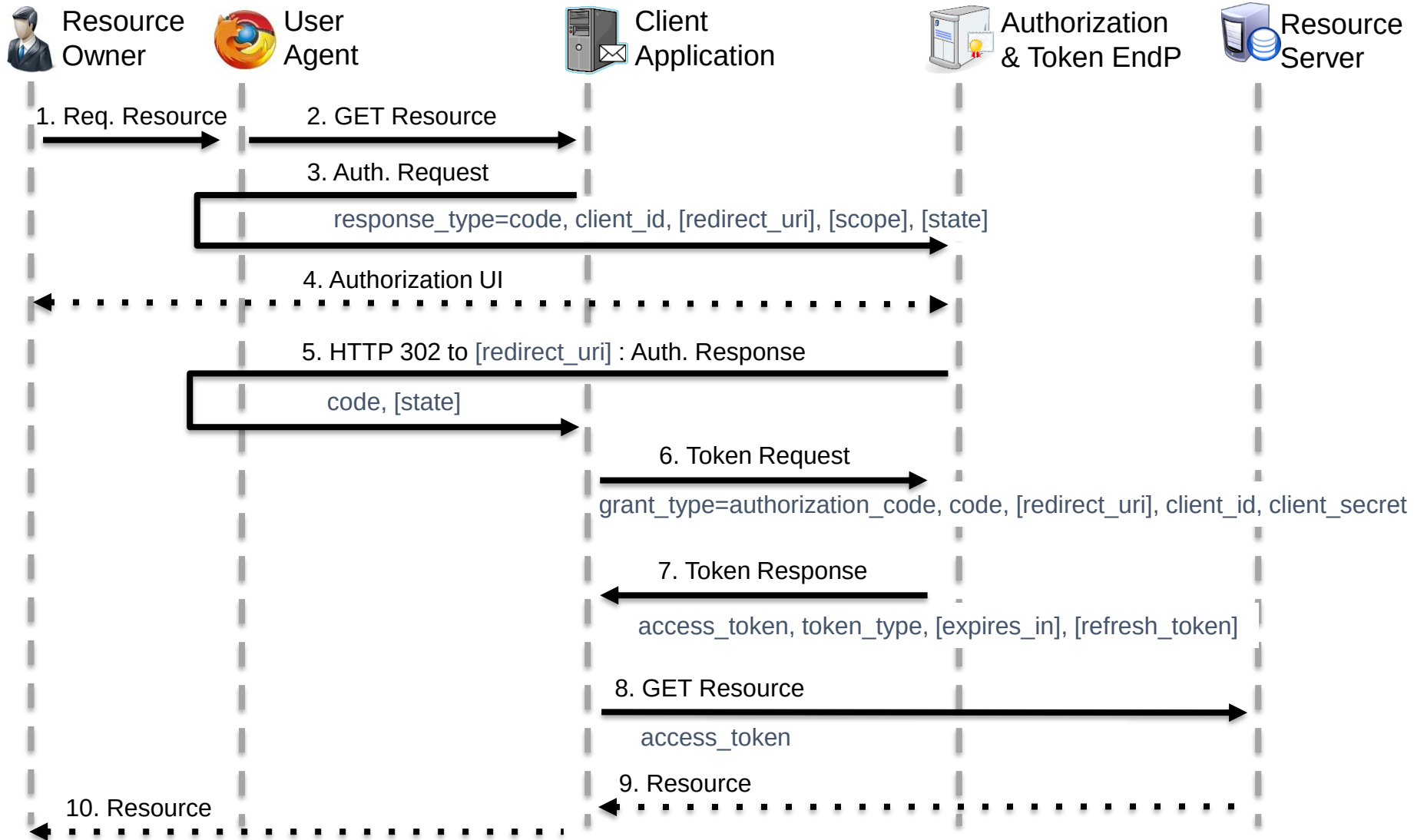
Overview

1. Motivation
2. Short History
3. OAuth vs. OpenID Connect
4. OpenID Connect Code Flow in Detail
-  5. Flows
6. Attacks
 1. Attacks on ID token
 2. Attacks on Other Parameters

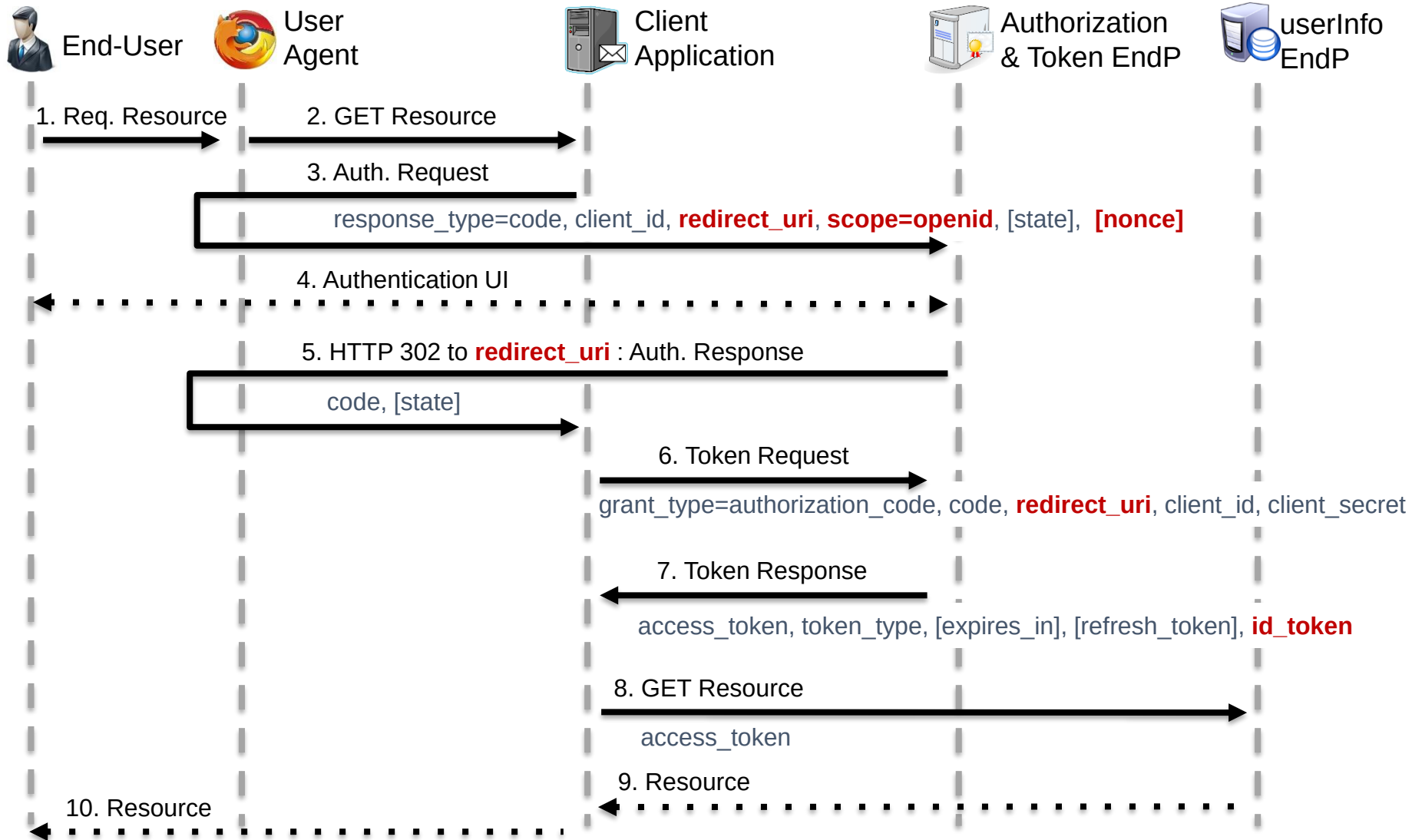
Flows

- Implicit flow deprecated for OAuth
- Hybrid flow not existent for OAuth
- We will present only code flow

OAuth: Authorization Code Grant



OIDC: Authentication Code Flow



Flow Recognition Cheat Sheet

- OpenID Connect has `scope=openid`
 - OAuth does not define any scope value
- Flow distinguished by `response_type`

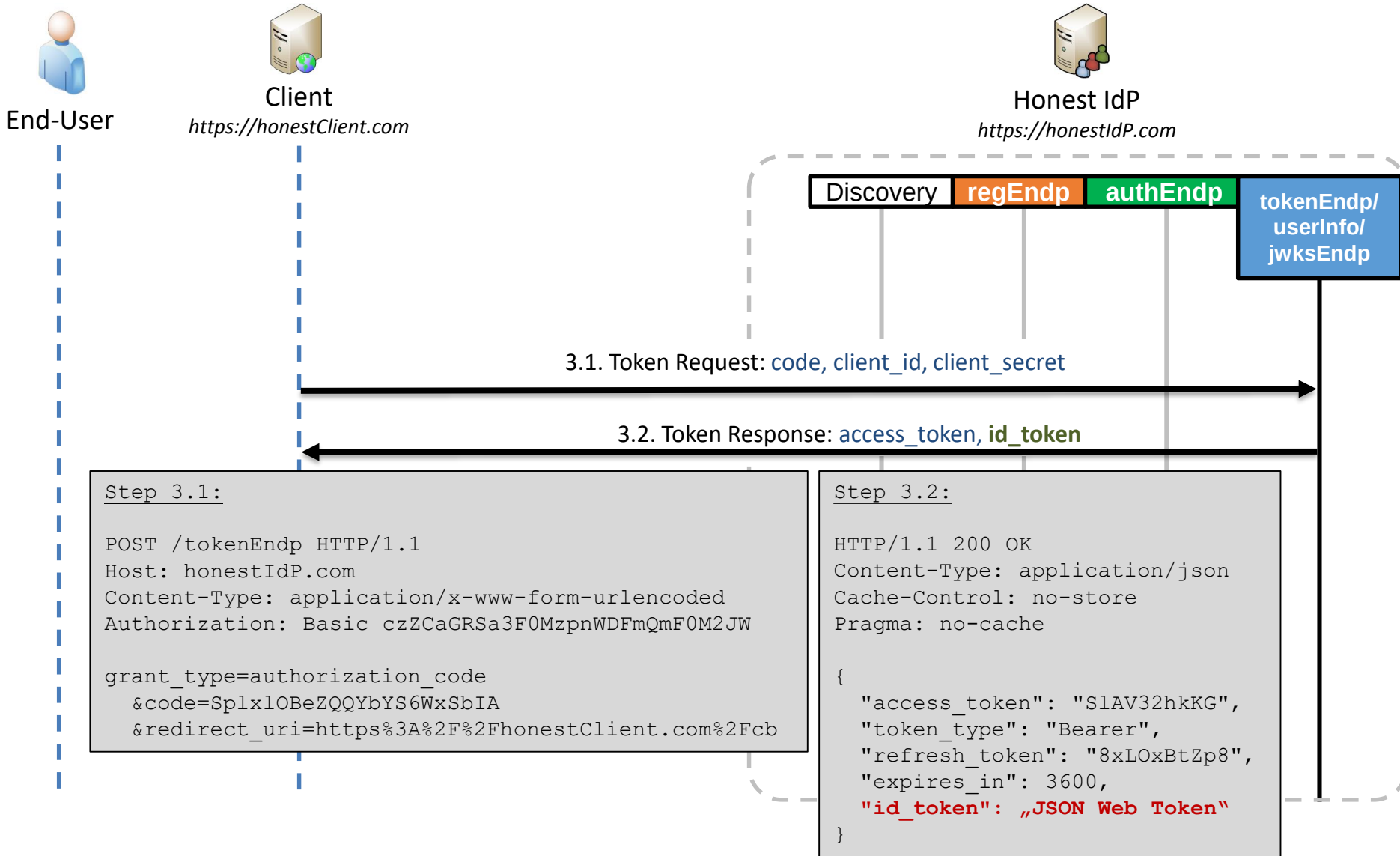
	OAuth	OpenID Connect
Code	<code>code</code>	<code>code</code>
Implicit	<code>token</code>	<code>id_token</code> <code>id_token token</code> (token means <code>access_token</code>)
Hybrid	(not existing)	<code>code id_token</code> <code>code token</code> <code>code id_token token</code>

Overview

1. Motivation
2. Short History
3. OAuth vs. OpenID Connect
4. OpenID Connect Code Flow in Detail
5. Flows
6. Attacks
 1. Attacks on ID token
 2. Attacks on Other Parameters



OIDC: id_token in Code Flow



SSO Token in OpenID Connect

Identity



JSON Web Token

```
eyJhbGciOiJIUzI1NiIsI  
nR5cCI6IkpXVCJ9.eyJle  
HAiOiJlEzODY4OTkxMzEsIm  
lzcyl6ImppcmE6MTU0ODk  
1OTUiLCJxc2giOiI4MDYz  
ZmY0Y2ExZTQxZGY3YmM5M  
GM4YWI2ZDBmNjIwN2Q0OT  
FjZjZkYWQ3YzY2ZWE3OTd  
iNDYxNGI3MTkyMmU5Iiw  
aWF0IjoxMzg2ODk4OTUxf  
Q.uKqU9dTB6gKwG6jQCuX  
YAImNdfNRw98Hw_IWuA5M  
aMo
```

Freshness



Recipient



Security



Decode with Base64 URL

SSO Token in OpenID Connect

Identity



=

iss

sub

JWT Header

```
{  
  "alg": "RS256",  
  "typ": "JWT"  
}
```

Freshness



=

iat

exp

nonce

JWT Body

```
{  
  "iss": "https://idp.com/",  
  "sub": "user1",  
  "iat": 1442673964,  
  "exp": 1444148308,  
  "nonce": "40c6b33b9a2e",  
  "aud": "client_id"  
}
```

Recipient



=

aud

JWT Signature

[0x02, 0xF1, ..., 0xDA]

Security

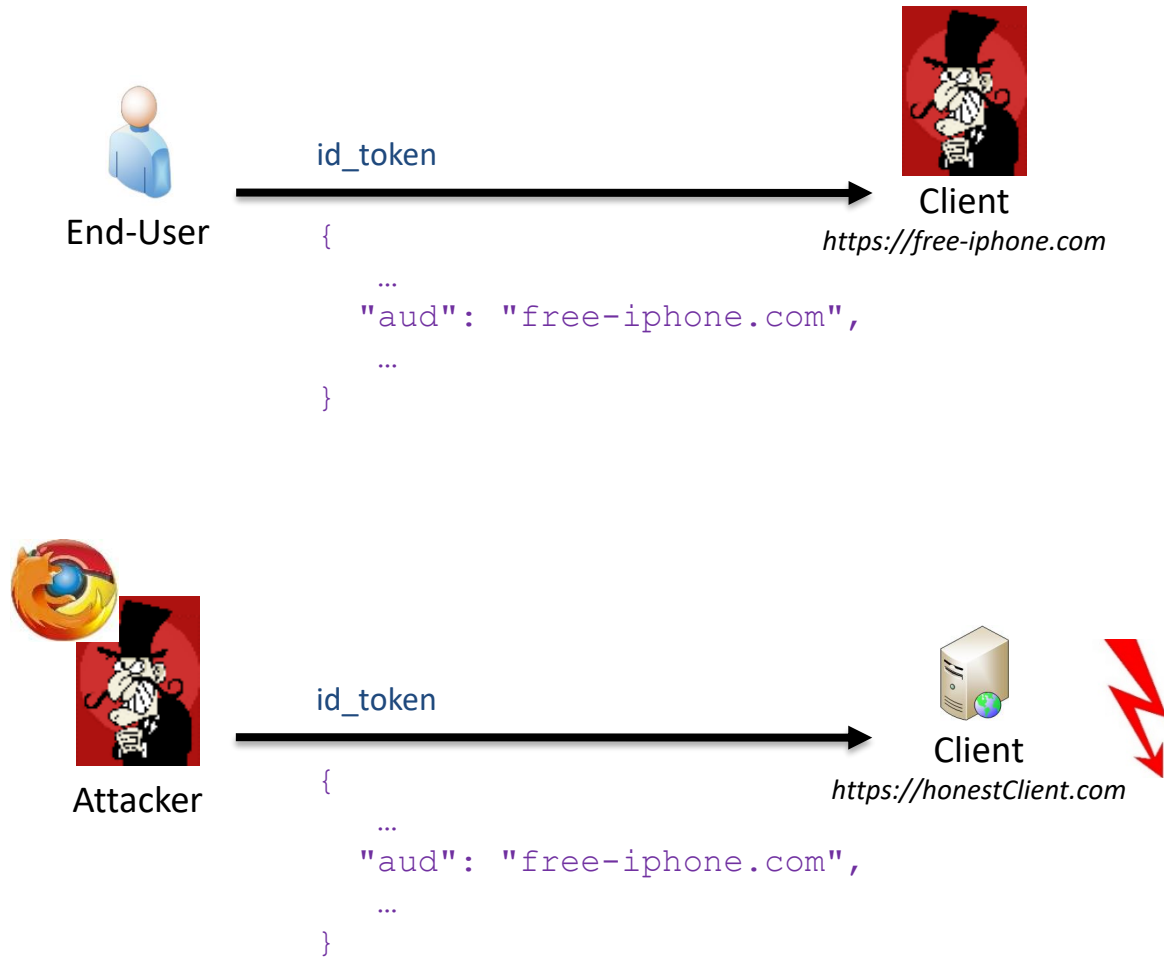


=

sig

alg

Attack: Wrong Recipient



Attack: Signature Bypass

Identity



=

iss

sub

Freshness



=

iat

exp

nonce

Recipient



=

aud

Security



=

sig

alg

JWT Header

```
{  
  "alg": "RS256",  
  "typ": "JWT"  
}
```

JWT Body

```
{  
  "iss": "https://idp.com/",  
  "sub": "user1",  
  "iat": 1442673964,  
  "exp": 1444148308,  
  "nonce": "40c6b33b9a2e",  
  "aud": "client_id"  
}
```

JWT Signature

```
[0x02, 0xF1, ..., 0xDA]
```

Attack: Signature Bypass

- alg defines Algorithm

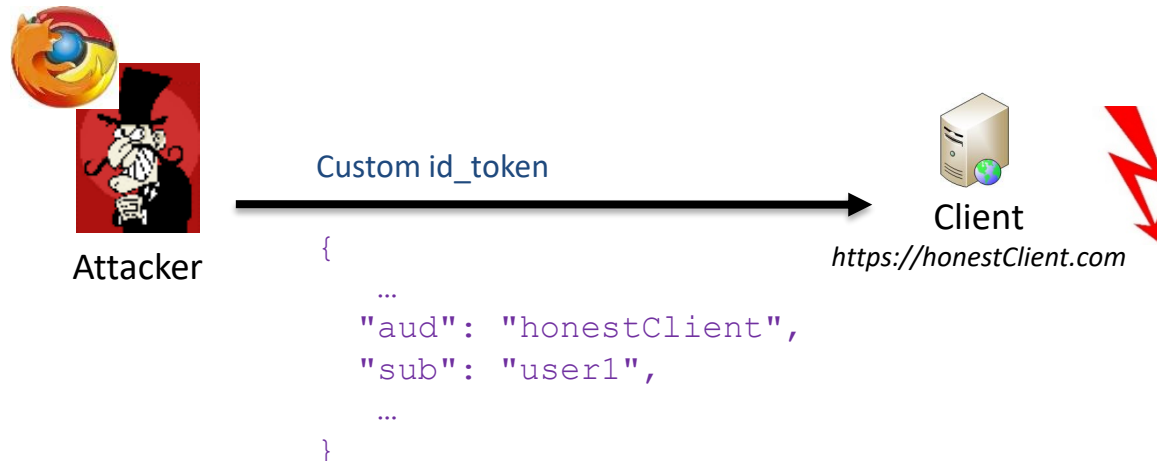
```
{  
  "alg": "RS256",  
  "typ": "JWT"  
}
```

alg Parameter Value	Digital Signature or MAC Algorithm	Implementation Requirements
HS256	HMAC using SHA-256	Required
HS384	HMAC using SHA-384	Optional
HS512	HMAC using SHA-512	Optional
RS256	RSASSA-PKCS-v1_5 using SHA-256	Recommended
RS384	RSASSA-PKCS-v1_5 using SHA-384	Optional
RS512	RSASSA-PKCS-v1_5 using SHA-512	Optional
ES256	ECDSA using P-256 and SHA-256	Recommended+
ES384	ECDSA using P-384 and SHA-384	Optional
ES512	ECDSA using P-521 and SHA-512	Optional
PS256	RSASSA-PSS using SHA-256 and MGF1 with SHA-256	Optional
PS384	RSASSA-PSS using SHA-384 and MGF1 with SHA-384	Optional
PS512	RSASSA-PSS using SHA-512 and MGF1 with SHA-512	Optional
none	No digital signature or MAC performed	Optional

Setting alg to "none" allows to remove the signature

Attack: Signature Bypass

- Signature Exclusion Attack:
 - Create a custom JWT
 - Configure `iss`, `sub`, `exp`, `iat`, `nonce`, `aud` as needed
 - Set `alg` to `none`
 - Set signature value to empty string



Overview

1. Motivation
2. Short History
3. OAuth vs. OpenID Connect
4. OpenID Connect Code Flow in Detail
5. Flows
6. Attacks
 1. Attacks on ID token
 2. Attacks on Other Parameters



CSRF

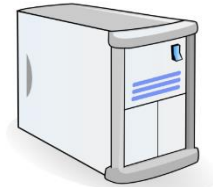
- Cross Site Request Forgery
- Forces victim to execute certain actions

Recap - CSRF

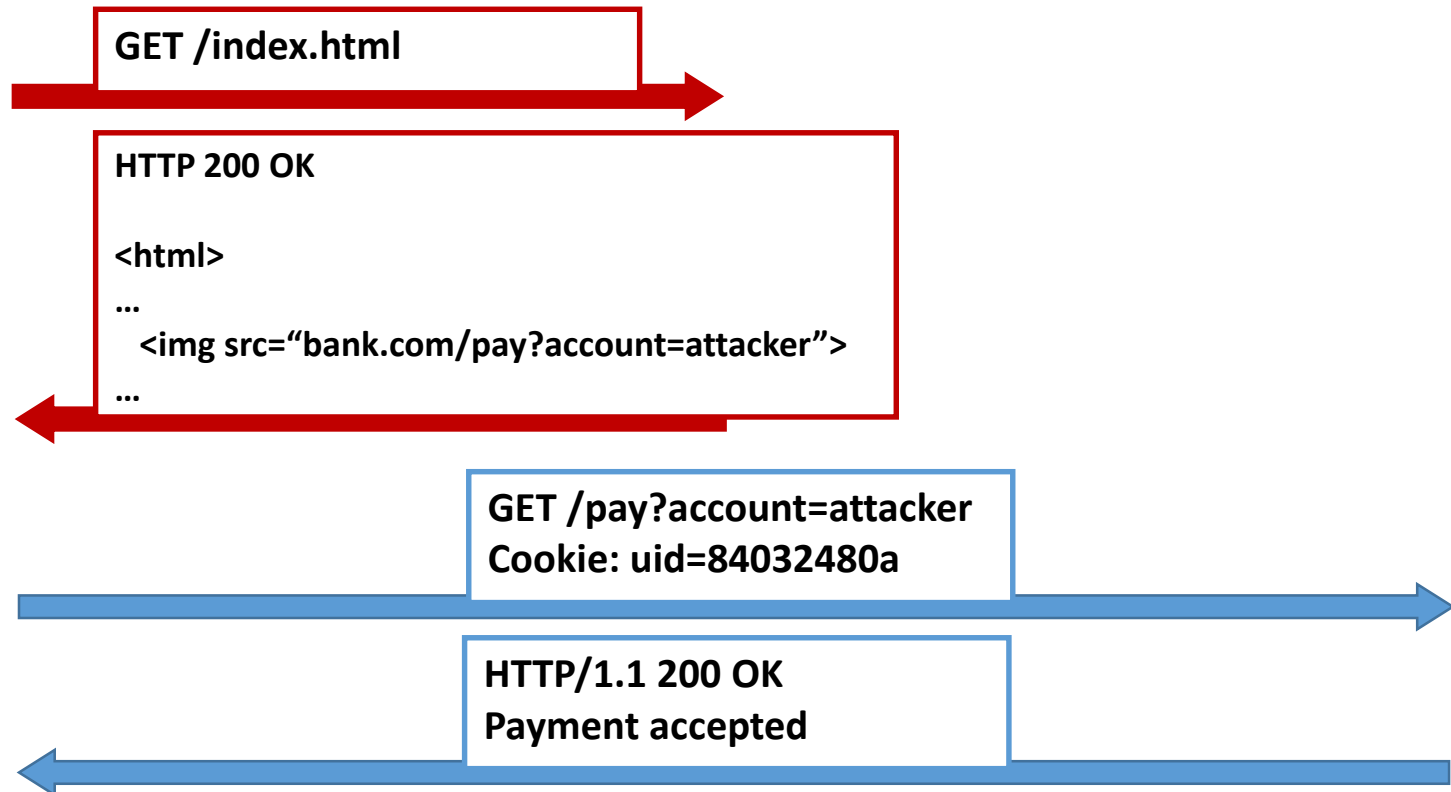


attacker.com

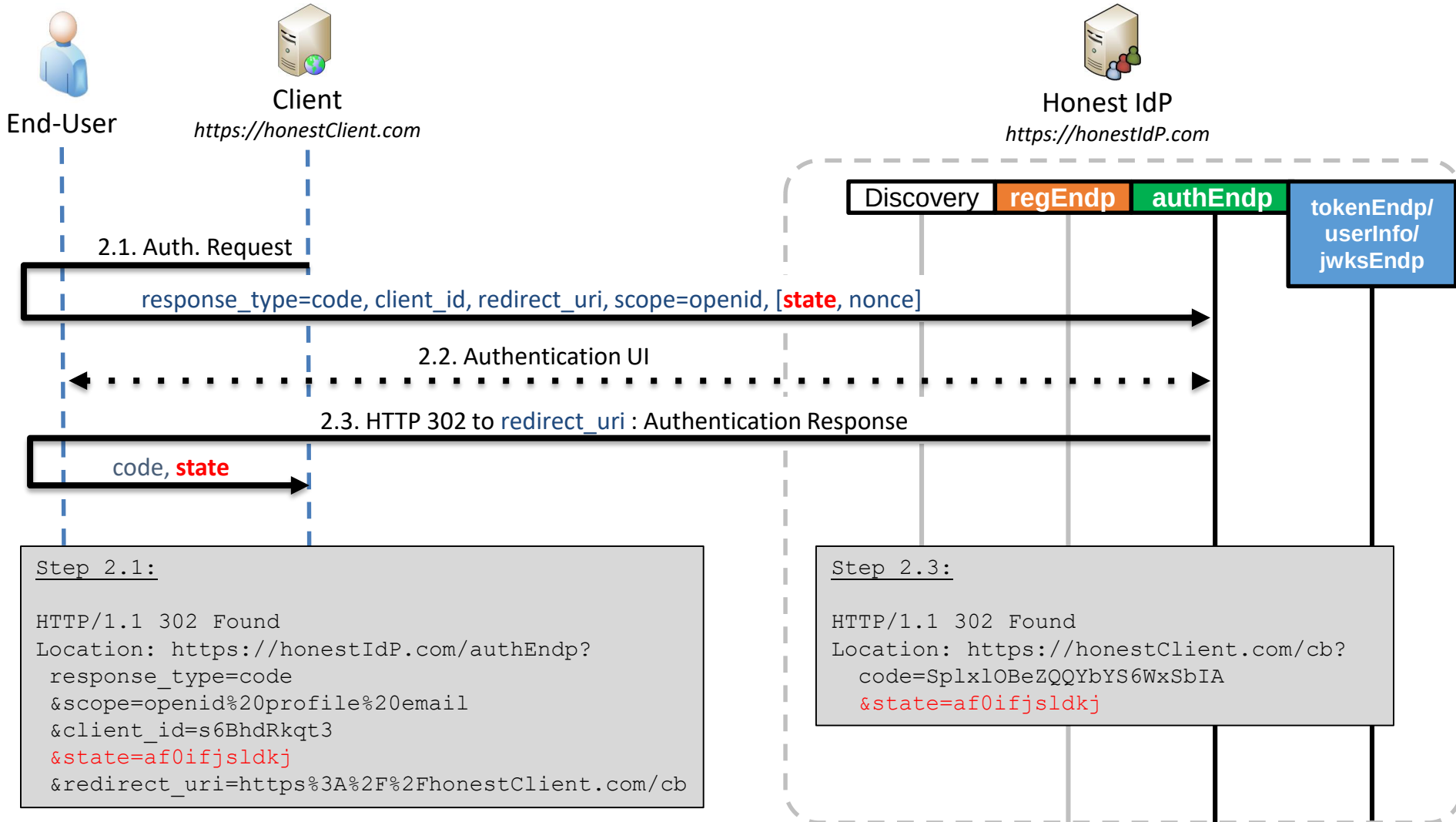
I have
dog
pictures!



Bank.com



CSRF Prevention Using state Parameter



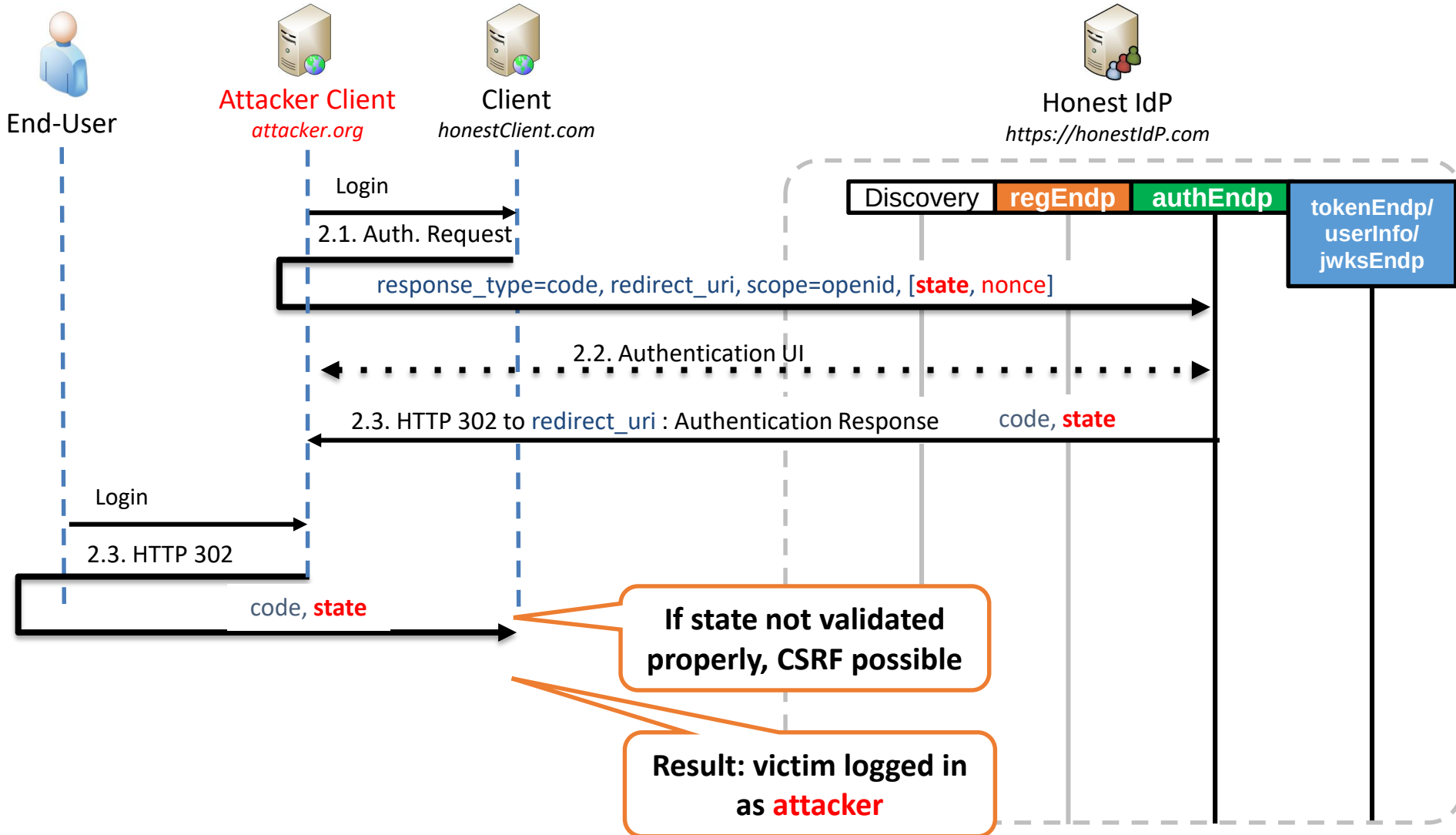
CSRF in OAuth/OIDC

- Typical web attack
 - But there are some "special" parameters to prevent it in OAuth / OIDC
- Two parameters: state, nonce
- Possible mistakes:
 - State, nonce are not used
 - State, nonce are set to a fixed value
 - State, nonce are not verified
 - State, nonce are not bound to the browser session

CSRF in OAuth/OIDC

- Attacker goals:
 - Log in the victim into victim's account
 - **Log in the victim into attacker's account**

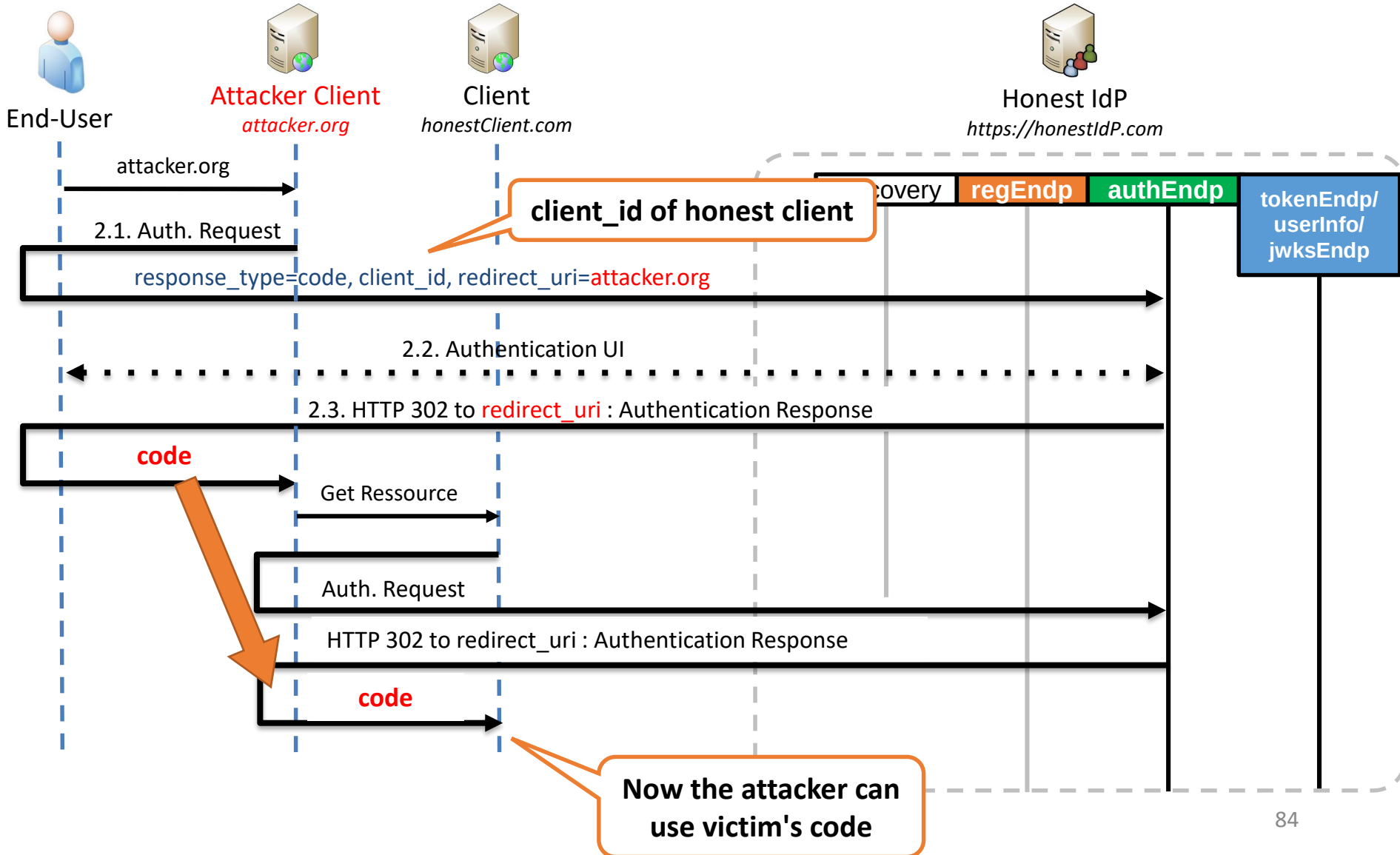
CSRF: Login Victim into Attacker's Account



Open / Covert Redirect

- Attacker goals:
 - Steal valid token (code, id_token, access_token)
 - Inject the token
- Requirement:
 - Victim clicks a link
- Idea:
 - Force the IdP to send valid tokens to attacker's domain
 - Possible if redirect_uri not checked properly
- Other possible parameters for similar attacks: next_url, next, goto, ...

Open Redirect: Code Flow



Redirect Countermeasure

- General problem: no validation of redirect_uri
 - IdP needs to store a list of client_id and redirect_uri pairs and validated them
- Possible solution:
 - Strict validation of redirect URIs
 - Usage of allow lists

Conclusions

- OAuth = Authorization
- OpenID Connect = Authentication
- Basic flow: Code
 - Many other flows: Implicit, Hybrid, Popup, ...
 - Many extensions: PKCE, IoT
- Attacks:
 - Web attacks applicable to both standards...have different variations and impact
- Next lecture: Rest Security
 - Please attend! 😊

Questions



Acknowledgements

- Tessa pictures by https://www.fiverr.com/tnhs_project



- Ömmes pictures by Janine.Plickert@rub.de



- The slides were provided by Christian Mainka, Vladislav Mladenov, and Karsten Meyer zu Selhausen

Sources & References

- https://openid.net/specs/openid-connect-core-1_0.html#CodeFlowAuth
- <https://oauth.net/>
- <https://openid.net/specs/specs-1.0.bml>
- <http://saml.xml.org/saml-specifications>
- <https://aaronparecki.com/2019/12/12/21/its-time-for-oauth-2-dot-1>
- <https://tools.ietf.org/html/rfc6749>
- <https://tools.ietf.org/html/rfc6750>
- <https://tools.ietf.org/html/rfc6819>
- <https://tools.ietf.org/html/draft-ietf-oauth-security-topics>

Sources & References

- SoK: Single Sign-On Security – An Evaluation of OpenID Connect
 - Christian Mainka, Vladislav Mladenov, Tobias Wich, Jörg Schwenk
 - IEEE European Symposium on Security and Privacy (EuroS&P 2017)
- <https://tools.ietf.org/html/rfc7519>
- OpenID Connect Token Validation
 - https://openid.net/specs/openid-connect-core-1_0.html#IDTokenValidation (Code Flow)
 - https://openid.net/specs/openid-connect-core-1_0.html#ImplicitIDTValidation (Implicit Flow)
 - https://openid.net/specs/openid-connect-core-1_0.html#HybridIDTValidation (Hybrid Flow/Front Channel)
 - https://openid.net/specs/openid-connect-core-1_0.html#HybridIDTValidation2 (Hybrid Flow/Back Channel)
- JWT Debugger: <https://jwt.io/>